



Facultad de Ingeniería

Carrera de Ingeniería Empresarial y de sistemas

INFORME TÉCNICO

" Análisis de Datos e Inteligencia de Negocios en E-Commerce "

Integrantes

Chavez Durand, Sofia

Jacay Ramos, Matias

Quispe Córdova, Andrea

Vasquez Ipanaque, Grace

Zarzosa Ramos, Hoberths

Dataset: OLIST E-Commerce

Curso: Bases de Datos y Big Data

Sección: 5D

Docente: Mg. Freud Enrique Melgar Aliaga

2026-I

Lima - Perú

INDICE

1.	Introducción	4
1.1.	Objetivo General	5
1.2.	Objetivos Específicos	5
2.	ETAPA 1 — Selección y Exploración del Dataset	6
2.1.	Descripción del Dataset	6
2.2.	Dominio de negocio.....	7
2.3.	Estructura de los datos	7
2.4.	Preguntas de negocio	11
2.5.	Hallazgos iniciales de la exploración de datos	11
3.	ETAPA 2 — Modelado y Diseño de la Base de Datos.....	12
3.1.	Entidades principales y sus atributos a partir del dataset	12
3.2.	Las relaciones entre entidades (cardinalidad: 1:1, 1:N, N:M)	14
3.3.	Diagrama Entidad-Relación (MER)	16
3.4.	Transformación del MER al Modelo Relacional	17
3.5.	Normalización (1FN-2FN-3FN)	18
3.6.	Script DDL con restricciones de integridad.....	21
4.	ETAPA 3 - Implementación y Carga de Datos / ETL	25
4.1.	Creación de la base de datos en Microsoft SQL Server	25
4.2.	Ejecución del script DDL y verificar la creación correcta de tablas	25
4.3.	Implementación del proceso ETL:.....	28
4.3.1.	Extracción: lectura del archivo CSV/XLSX del dataset	28
4.3.2.	Transformación y limpieza de datos.....	29
4.3.3.	Carga: inserción de datos en la base de datos.	30
4.4.	Número de registros cargados por tabla y los errores encontrados	37
4.5.	Reporte de calidad de datos.....	40
5.	ETAPA 4 - Consultas SQL y Análisis Avanzado	43
5.1.	Desarrollo de consultas para la toma de decisiones	43
5.2.	Desarrollo de consultas	46
6.	ETAPA 5 - Big Data e Integración de Resultados.....	50
6.1.	Análisis de las 5 V del Big Data	50

6.2. Implementación de herramientas de Big Data	53
6.2.1. Apache Spark.....	54
6.2.2. Google BigQuery	54
6.3. Consulta en PySpark vs Pandas.....	55
6.3.1. Consulta 1: Conteo por estado de pedido	56
6.3.2. Consulta 2: Top 5 fechas con mayor volumen.....	56
6.3.3. Análisis comparativo	56
6.4. Dashboard en Power BI.....	57
7. Conclusiones	58
8. Anexo.....	59
8.1. Conformación de Equipos y Roles	59
9. Referencias Bibliográficas	60

1. Introducción

El presente proyecto integrador ha sido desarrollado en el marco del curso de Bases de Datos y Big Data con el propósito de aplicar los conocimientos adquiridos en modelado de bases de datos, normalización, lenguaje SQL y análisis de información empresarial.

Para el desarrollo del proyecto se empleará el dataset OLIST E-Commerce Public Dataset, un conjunto de datos de comercio electrónico que contiene información sobre clientes, pedidos, productos, vendedores, pagos y valoraciones realizadas por los usuarios. Este dataset permite analizar el comportamiento de compra de los clientes, el rendimiento de las ventas y diversos indicadores relacionados con la gestión comercial de una plataforma de comercio electrónico.

Mediante la aplicación de técnicas de modelado de datos, normalización, procesos ETL y consultas SQL avanzadas, se busca obtener información relevante sobre las ventas, el comportamiento de los clientes, el desempeño de los vendedores, los productos más demandados y los niveles de satisfacción de los usuarios.

Asimismo, el proyecto permitirá desarrollar habilidades prácticas relacionadas con la administración de bases de datos y el análisis de información, contribuyendo a la formación profesional en el área de ingeniería y gestión de datos.

La base de datos diseñada permitirá almacenar, organizar y relacionar la información proveniente del dataset mediante una estructura relacional optimizada, facilitando la ejecución de consultas y análisis orientados a la toma de decisiones empresariales.

Las principales tablas que conforman la base de datos son:

- ❖ Clientes: Almacena la información de los clientes, incluyendo identificadores únicos y datos geográficos como ciudad y estado.
- ❖ Pedidos: Contiene la información de los pedidos realizados por los clientes, su estado y las fechas asociadas al proceso de compra y entrega.
- ❖ Detalle_Pedidos: Registra los productos incluidos en cada pedido, así como el vendedor responsable, el precio del producto y el costo del envío.
- ❖ Productos: Almacena información relacionada con las características de los productos comercializados y sus respectivas categorías.
- ❖ Vendedores: Contiene los datos de los vendedores que participan en la plataforma de comercio electrónico.
- ❖ Pagos: Registra los métodos de pago utilizados por los clientes, el número de cuotas y el monto pagado por cada pedido.
- ❖ Valoraciones: Almacena las calificaciones y comentarios realizados por los clientes sobre su experiencia de compra y entrega.

- ❖ Geolocalización: Centraliza la información geográfica (ciudad, estado y código postal) de compradores y comercios para asegurar la correcta normalización del modelo (3FN).

La integración de estas entidades permitirá evaluar el rendimiento de las ventas, el comportamiento del consumidor y el desempeño logístico, contribuyendo directamente al desarrollo de habilidades analíticas orientadas a la ingeniería y gestión de datos a gran escala.

1.1. Objetivo General

Diseñar, implementar y administrar una base de datos relacional basada en el dataset OLIST E-Commerce, aplicando técnicas de modelado, normalización, procesos ETL y consultas SQL avanzadas para generar información estratégica que contribuya a la toma de decisiones empresariales.

1.2. Objetivos Específicos

- Explorar y comprender la estructura del dataset OLIST E-Commerce, identificando entidades, atributos, tipos de datos y relaciones existentes entre las tablas.
- Diseñar el Modelo Entidad-Relación (MER) y el Modelo Relacional correspondiente, aplicando las tres formas normales (1FN, 2FN y 3FN) para garantizar la integridad y consistencia de los datos.
- Implementar la base de datos en SQL SERVER mediante la creación de tablas, claves primarias, claves foráneas y restricciones de integridad.
- Realizar el proceso ETL (Extracción, Transformación y Carga) para preparar, limpiar y cargar los datos provenientes del dataset a la base de datos relacional.
- Desarrollar consultas SQL de nivel intermedio y avanzado utilizando filtros, agrupaciones, funciones de agregación, JOINS, subconsultas, vistas y procedimientos almacenados.
- Analizar los resultados obtenidos mediante las consultas SQL para identificar patrones de compra, tendencias de ventas y oportunidades de mejora en la gestión comercial.
- Evaluar el desempeño de vendedores, productos y métodos de pago a través del análisis de los datos almacenados en la base de datos.
- Analizar el dataset bajo las dimensiones del Big Data (Volumen, Variedad, Velocidad, Veracidad y Valor), determinando su potencial para el procesamiento y análisis a gran escala.
- Investigar herramientas de Big Data que permitan procesar y analizar grandes volúmenes de información de comercio electrónico de manera eficiente.

- Generar conclusiones y recomendaciones basadas en los resultados obtenidos para apoyar la toma de decisiones dentro de un entorno empresarial.

2. ETAPA 1 — Selección y Exploración del Dataset

2.1. Descripción del Dataset

Nombre del Dataset

Conjunto de Datos Público de Comercio Electrónico Brasileño por Olist (Brazilian E-Commerce Public Dataset by Olist).

Origen Institucional

El conjunto de datos fue obtenido de la plataforma Kaggle, donde fue publicado por Olist, una empresa brasileña especializada en soluciones de comercio electrónico que conecta pequeños y medianos vendedores con diversos marketplaces. Su publicación forma parte de una iniciativa orientada a fomentar el aprendizaje, la investigación y el desarrollo de proyectos en áreas como ciencia de datos, análisis de negocios, minería de datos y bases de datos.

El dataset reúne información relacionada con pedidos, clientes, vendedores, productos, pagos, reseñas y procesos logísticos generados dentro del ecosistema de comercio electrónico de Olist.

Asimismo, es importante mencionar que Olist es una organización privada y no una entidad gubernamental. Por ello, el conjunto de datos no proviene de organismos públicos ni de portales oficiales de datos abiertos, sino de una empresa del sector privado que decidió compartir información anonimizada para contribuir al desarrollo de la comunidad académica y profesional. Además, los datos publicados cumplen con criterios de anonimización y protección de la privacidad, garantizando que no sea posible identificar directamente a clientes, vendedores o socios comerciales.

Año de Publicación

Los datos corresponden a transacciones realizadas entre los años 2016 y 2018.

Fuente

Kaggle – Brazilian E-Commerce Public Dataset by Olist.

2.2. Dominio de negocio

El dataset pertenece al sector de Comercio Electrónico (E-Commerce) y Ventas Minoristas Digitales.

Las operaciones registradas incluyen:

- Gestión de clientes.
- Pedidos de compra.
- Productos comercializados.
- Vendedores.
- Pagos.
- Logística y entregas.
- Valoraciones de clientes.
- Información geográfica.

2.3. Estructura de los datos

- **Tabla: olist_clientes**

Columna	Tipo de Dato	Llave	Nulo	Descripción
id_cliente	VARCHAR(50)	PK	NO	Identificador único del cliente asociado a un pedido específico.
id_unico_cliente	VARCHAR(50)		NO	Identificador único de la persona física (permite rastrear compras recurrentes).
codigo_postal	VARCHAR(10)	FK	NO	Primeros 5 dígitos del código postal del cliente (CEP brasileño).

- **Tabla: olist_productos**

Columna	Tipo de Dato	Llave	Nulo	Descripción
id_producto	VARCHAR(50)	PK	NO	Identificador alfanumérico único del producto.
categoria	VARCHAR(100)		SÍ	Nombre de la categoría a la que pertenece el producto.
peso_g	INT		SÍ	Peso físico del producto empaquetado, en gramos.
longitud_cm	INT		SÍ	Largo del empaque del producto, en centímetros.

altura_cm	INT		SÍ	Alto del empaque del producto, en centímetros.
ancho_cm	INT		SÍ	Ancho del empaque del producto, en centímetros.

- **Tabla: olist_vendedores**

Columna	Tipo de Dato	Llave	Nulo	Descripción
id_vendedor	VARCHAR(50)	PK	NO	Identificador alfanumérico único del comercio/vendedor.
codigo_postal	VARCHAR(10)	FK	NO	Primeros 5 dígitos del código postal del vendedor.

- **Tabla: olist_pedidos**

Columna	Tipo de Dato	Llave	Nulo	Descripción
id_pedido	VARCHAR(50)	PK	NO	Identificador único de la orden de compra.
id_cliente	VARCHAR(50)	FK	NO	ID del comprador (Se relaciona con olist_clientes).
estado_pedido	VARCHAR(20)		NO	Estado actual de la orden (ej. entregado, enviado, cancelado).
fecha_compra	DATETIME		NO	Fecha y hora exacta en la que el cliente confirmó el pedido.
fecha_aprobacion	DATETIME		SÍ	Fecha y hora en la que se confirmó el pago del pedido.
fecha_entrega_transportista	DATETIME		SÍ	Fecha y hora en la que el paquete fue entregado al courier logístico.
fecha_entrega_cliente	DATETIME		SÍ	Fecha y hora real en la que el cliente recibió el paquete.

fecha_estimada_entrega	DATETIME		SÍ	Fecha prometida al cliente en el momento de la compra.
------------------------	----------	--	----	--

- **Tabla: olist_detalle_pedidos**

Columna	Tipo de Dato	Llave	Nulo	Descripción
id_pedido	VARCHAR(50)	PK / FK	NO	ID del pedido general (Se relaciona con olist_pedidos).
id_item_pedido	INT	PK	NO	Número secuencial (1, 2, 3...) del artículo dentro del mismo pedido.
id_producto	VARCHAR(50)	FK	NO	ID del artículo comprado (Se relaciona con olist_productos).
id_vendedor	VARCHAR(50)	FK	NO	ID del comercio que lo vende (Se relaciona con olist_vendedores).
fecha_limite_envio	DATETIME		NO	Fecha máxima que tiene el vendedor para entregar el producto al courier.
precio	DECIMAL(10,2)		NO	Precio de venta del producto (sin incluir flete).
valor_flete	DECIMAL(10,2)		NO	Costo del transporte (envío) de ese ítem específico.

- **Tabla: olist_pagos_pedido**

Columna	Tipo de Dato	Llave	Nulo	Descripción
id_pago	VARCHAR(50)	PK	NO	Llave primaria autoincremental para optimizar consultas (Añadido por diseño).
id_pedido	VARCHAR(50)	FK	NO	ID del pedido al que pertenece el pago (Se relaciona con olist_pedidos).

secuencia_pago	INT		NO	Orden del pago en caso de usar múltiples métodos (1 = primer método, etc.).
tipo_pago	VARCHAR(50)		NO	Método utilizado (ej. credit_card, boleto, voucher, debit_card).
cuotas_pago	INT		NO	Cantidad de meses/cuotas en las que se dividió el pago.
valor_pago	DECIMAL(10,2)		NO	Monto total procesado en esta transacción específica.

- **Tabla: olist_valoraciones**

Columna	Tipo de Dato	Llave	Nulo	Descripción
id_valoraciones	VARCHAR(50)	PK	NO	Identificador único de la reseña.
id_pedido	VARCHAR(50)	FK	NO	ID del pedido que está siendo evaluado (Se relaciona con olist_pedidos).
puntaje	INT		NO	Calificación otorgada por el cliente, en una escala del 1 al 5.
mensaje_comentario	TEXT		SÍ	Cuerpo de texto con la opinión escrita por el cliente.
fecha_creacion	DATETIME		NO	Fecha y hora en la que se envió la encuesta de satisfacción al cliente.
fecha_respuesta	DATETIME		SÍ	Fecha y hora en la que el cliente completó y envió la reseña.

- **Tabla: olist_geolocalizacion**

Columna	Tipo de Dato	Llave	Nulo	Descripción
codigo_postal	VARCHAR(10)	PK	No	Identificador único de la zona postal. Funciona como la clave principal para relacionar la ubicación geográfica con el resto de las entidades del sistema.

ciudad	VARCHAR(100)	No	No	Nombre completo de la ciudad, municipio o localidad asociada al código postal.
estado	CHAR(2)	No	No	Sigla o abreviatura de dos caracteres correspondiente al estado o unidad federativa (ej. 'SP', 'RJ').

Posibles inconsistencias

- Pedidos sin fecha de entrega.
- Productos con dimensiones faltantes.
- Reseñas sin comentarios escritos.
- Pedidos cancelados con información parcial.
- Diferencias entre fecha estimada y fecha real de entrega.

2.4. Preguntas de negocio

A partir de del análisis preliminar se plantean las siguientes preguntas:

- ¿Cuáles son las categorías de productos con mayor volumen de ventas?
- ¿Qué estados o ciudades generan la mayor cantidad de pedidos?
- ¿Existe relación entre el tiempo de entrega y la satisfacción del cliente?
- ¿Qué vendedores registran mayores ingresos por ventas?
- ¿Qué porcentaje de pedidos fue entregado antes de la fecha estimada?

2.5. Hallazgos iniciales de la exploración de datos

- La información permite analizar ventas, logística y satisfacción del cliente.
- El modelo posee suficientes entidades para implementar una base de datos relacional normalizada hasta Tercera Forma Normal (3FN).
- Un pedido puede contener varios productos.
- Un pedido puede involucrar diferentes vendedores.
- El sistema está orientado a procesos completos de comercio electrónico.

3. ETAPA 2 — Modelado y Diseño de la Base de Datos

3.1. Entidades principales y sus atributos a partir del dataset

A partir del análisis del dataset OLIST E-Commerce, se identificaron 8 entidades principales, cada una representando un conjunto de datos con significado propio dentro del sistema de comercio electrónico.

Entidad 1: olist_clientes

Almacena la información de los clientes asociados a cada pedido realizado en la plataforma.

Atributo	Tipo de Dato	Descripción
id_cliente (PK)	VARCHAR(50)	Identificador único del cliente por pedido
id_unico_cliente	VARCHAR(50)	Identificador de la persona física (permite rastrear compras recurrentes)
codigo_postal	VARCHAR(10)	Primeros 5 dígitos del código postal del cliente

Entidad 2: olist_pedidos

Registra cada orden de compra realizada en la plataforma, incluyendo su estado y las fechas del ciclo de vida del pedido.

Atributo	Tipo de Dato	Descripción
id_pedido (PK)	VARCHAR(50)	Identificador único de la orden de compra
id_cliente (FK)	VARCHAR(50)	Cliente que realizó el pedido
estado_pedido	VARCHAR(20)	Estado actual de la orden (entregado, enviado, cancelado, etc.)
fecha_compra	DATETIME	Fecha y hora en que el cliente confirmó el pedido
fecha_aprobacion	DATETIME	Fecha y hora en que se confirmó el pago
fecha_entrega_transportista	DATETIME	Fecha en que el paquete fue entregado al courier
fecha_entrega_cliente	DATETIME	Fecha real en que el cliente recibió el paquete
fecha_estimada_entrega	DATETIME	Fecha prometida al cliente al momento de la compra

Entidad 3: olist_detalle_pedidos

Contiene los productos incluidos en cada pedido, con el vendedor responsable, precio y costo de envío de cada ítem.

Atributo	Tipo de Dato	Descripción
id_pedido (PK, FK)	VARCHAR(50)	Pedido al que pertenece el ítem
id_item_pedido (PK)	INT	Número secuencial del artículo dentro del pedido
id_producto (FK)	VARCHAR(50)	Producto comprado
id_vendedor (FK)	VARCHAR(50)	Vendedor responsable del ítem
fecha_limite_envio	DATETIME	Fecha máxima para que el vendedor entregue al courier
precio	DECIMAL(10,2)	Precio de venta del producto (sin flete)
valor_flete	DECIMAL(10,2)	Costo del envío del ítem

Entidad 4: olist_productos

Almacena las características físicas y de categoría de cada producto disponible en la plataforma.

Atributo	Tipo de Dato	Descripción
id_producto (PK)	VARCHAR(50)	Identificador único del producto
categoria	VARCHAR(100)	Categoría a la que pertenece el producto
peso_g	INT	Peso del producto empaquetado en gramos
longitud_cm	INT	Largo del empaque en centímetros
altura_cm	INT	Alto del empaque en centímetros
ancho_cm	INT	Ancho del empaque en centímetros

Entidad 5: olist_vendedores

Contiene los datos de los comercios y vendedores que operan dentro de la plataforma.

Atributo	Tipo de Dato	Descripción
id_vendedor (PK)	VARCHAR(50)	Identificador único del vendedor
codigo_postal	VARCHAR(10)	Código postal desde donde opera el vendedor

Entidad 6: olist_pagos_pedido

Registra los métodos y montos de pago utilizados por los clientes. Un pedido puede tener más de un método de pago.

Atributo	Tipo de Dato	Descripción
----------	--------------	-------------

id_pago (PK)	VARCHAR(100)	Identificador único del registro de pago
id_pedido (FK)	VARCHAR(50)	Pedido al que corresponde el pago
secuencia_pago	INT	Orden del método en caso de pagos múltiples
tipo_pago	VARCHAR(50)	Método utilizado (credit_card, boleto, voucher, etc.)
cuotas_pago	INT	Cantidad de cuotas en que se dividió el pago
valor_pago	DECIMAL(10,2)	Monto total procesado en esta transacción

Entidad 7: olist_valoraciones

Almacena las calificaciones y comentarios que los clientes realizan sobre su experiencia de compra.

Atributo	Tipo de Dato	Descripción
id_valoraciones (PK)	VARCHAR(50)	Identificador único de la reseña
id_pedido (FK)	VARCHAR(50)	Pedido que está siendo evaluado
puntaje	INT	Calificación del cliente (escala del 1 al 5)
mensaje_comentario	TEXT	Texto con la opinión del cliente
fecha_creacion	DATETIME	Fecha en que se envió la encuesta al cliente
fecha_respuesta	DATETIME	Fecha en que el cliente completó la reseña

Entidad 8: olist_geolocalizacion

Durante el análisis del dataset se identificó que las tablas CLIENTES y VENDEDORES compartían los mismos atributos de ubicación geográfica: codigo_postal, ciudad y estado. Esta repetición de información generaba redundancia en el modelo, ya que los mismos datos de una zona postal quedaban almacenados en dos tablas distintas. Para resolver esta situación, se creó la tabla GEOLOCALIZACION como una entidad independiente que centraliza la información geográfica y a la cual tanto CLIENTES como VENDEDORES hacen referencia a través del campo codigo_postal. Esta tabla fue incorporada como resultado del proceso de normalización, de la Tercera Forma Normal.

Atributo	Tipo de Dato	Descripción
codigo_postal (PK)	VARCHAR(10)	Identificador único de la zona postal
ciudad	VARCHAR(100)	Nombre de la ciudad o localidad asociada al código postal
estado	CHAR(2)	Sigla del estado o unidad federativa (ej. SP, RJ)

3.2. Las relaciones entre entidades (cardinalidad: 1:1, 1:N, N:M)

Una vez identificadas las entidades del dataset OLIST, se procedió a definir las relaciones existentes entre ellas, estableciendo la cardinalidad de cada vínculo en función de la lógica del negocio de la plataforma de comercio electrónico.

Entidad origen	Entidad destino	Cardinalidad
olist_clientes	olist_pedidos	1:N
olist_pedidos	olist_detalle_pedidos	1:N
olist_productos	olist_detalle_pedidos	1:N
olist_vendedores	olist_detalle_pedidos	1:N
olist_pedidos	olist_pagos_pedido	1:N
olist_pedidos	olist_valoraciones	1:N
olist_geolocalizacion	olist_clientes	1:N
olist_geolocalizacion	olist_vendedores	1:N

olist_clientes - olist_pedidos

Cardinalidad 1:N.- Un cliente puede realizar múltiples pedidos a lo largo del tiempo en la plataforma. Sin embargo, cada pedido registrado queda vinculado de forma exclusiva a un único cliente, lo que permite rastrear el historial de compras individual de cada usuario.

olist_pedidos - olist_detalle_pedidos

Cardinalidad 1:N.- Un pedido puede estar compuesto por uno o varios ítems, dado que un cliente puede adquirir múltiples productos dentro de una misma orden de compra, y cada ítem registrado en el detalle pertenece únicamente a un pedido específico.

olist_productos - olist_detalle_pedidos

Cardinalidad 1:N.- Un mismo producto puede aparecer en múltiples ítems de distintos pedidos realizados por diferentes clientes, y cada registro de detalle hace referencia a un único producto identificado dentro del catálogo.

olist_vendedores - olist_detalle_pedidos

Cardinalidad 1: N.- Un vendedor puede ser responsable de la venta de múltiples ítems distribuidos en distintos pedidos, y cada ítem de detalle está asociado a un único vendedor que gestiona ese artículo específico.

olist_pedidos - olist_pagos_pedido

Cardinalidad 1:N.- Un pedido puede ser abonado mediante más de un método de pago de forma simultánea, por ejemplo, combinando tarjeta de crédito con voucher, y cada registro de pago corresponde siempre a un único pedido.

olist_pedidos - olist_valoraciones

Cardinalidad 1:N.- Una vez completado el pedido, el cliente tiene la posibilidad de registrar una o más valoraciones sobre su experiencia de compra, y cada valoración queda vinculada exclusivamente al pedido que la originó.

olist_geolocalizacion - olist_clientes

Cardinalidad: 1:N Un mismo código postal puede estar asociado a múltiples clientes, y cada cliente referencia un único código postal que determina su ubicación geográfica.

olist_geolocalizacion - olist_vendedores

Cardinalidad: 1:N Un mismo código postal puede corresponder a múltiples vendedores, y cada vendedor opera desde una única zona postal registrada en la tabla de geolocalización.

3.3. Diagrama Entidad-Relación (MER)

Para representar la estructura lógica de la base de datos, se elaboró el Diagrama Entidad-Relación utilizando la notación Crow's Foot y Chen, las cuales permiten visualizar de forma clara las entidades, sus atributos y las relaciones existentes entre ellas junto con su cardinalidad. Los diagramas reflejan las 8 entidades identificadas a partir del dataset OLIST y las relaciones definidas en el punto anterior.

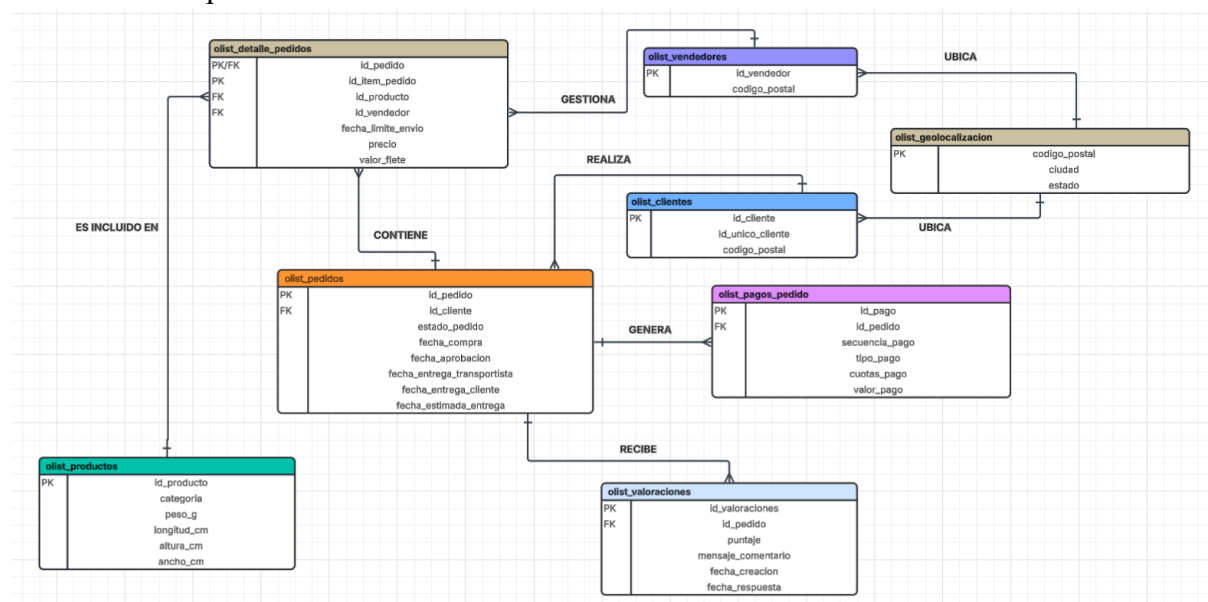


Figura 1. Modelo Entidad-Relación (MER) del sistema OLIST E-Commerce.

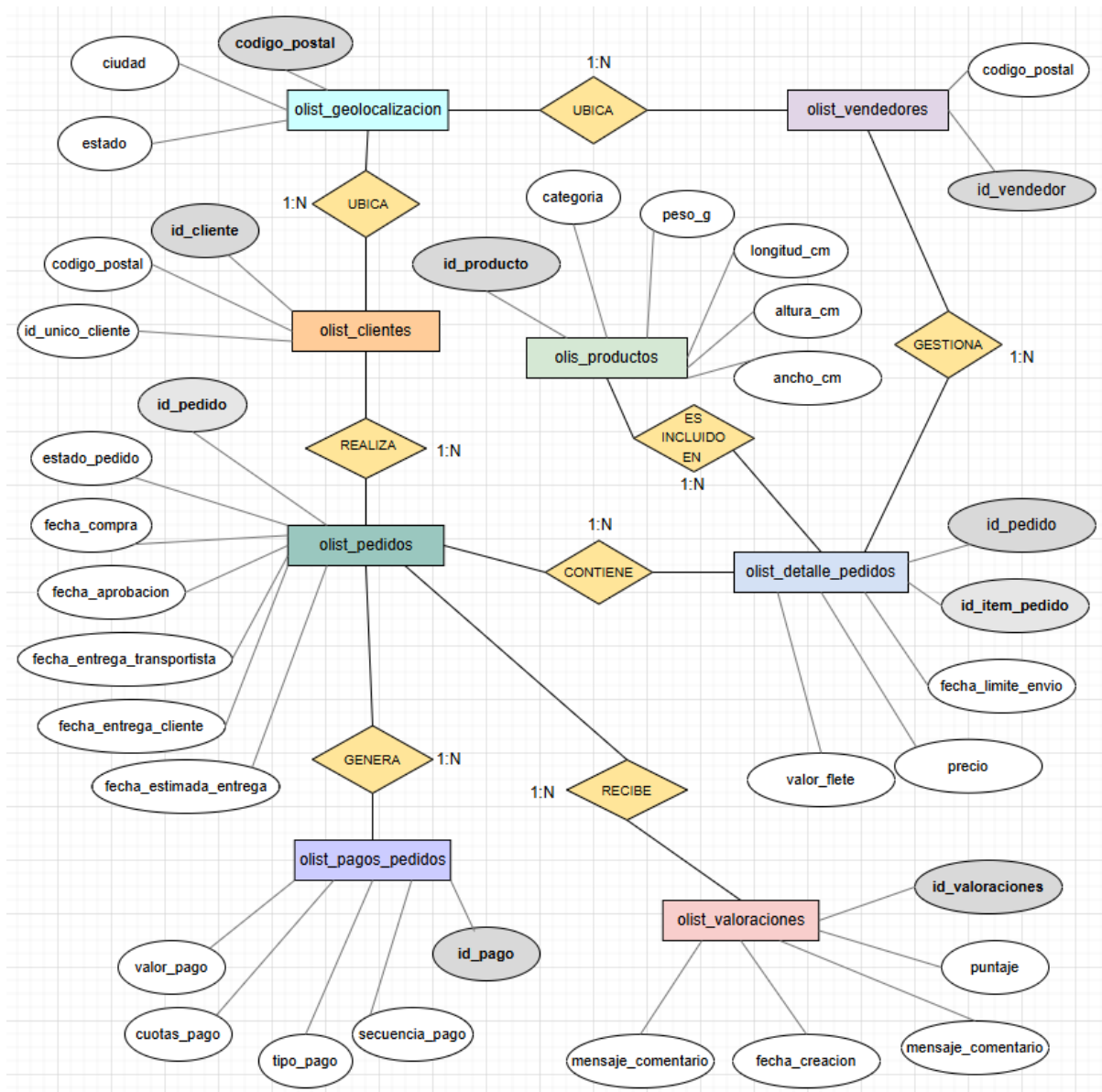


Figura 2. Modelo Entidad-Relación (MER) del sistema OLIST E-Commerce.

3.4. Transformación del MER al Modelo Relacional

Una vez elaborado el Diagrama Entidad-Relación, se procedió a transformarlo al Modelo Relacional, en el cual cada entidad se representa como una tabla con sus respectivos atributos, identificando las llaves primarias (PK) y las llaves foráneas (FK) que establecen las relaciones entre tablas.

olist_clientes: (id_cliente PK, id_unico_cliente, codigo_postal FK)

olist_pedidos: (id_pedido PK, id_cliente FK, estado_pedido, fecha_compra, fecha_aprobacion, fecha_entrega_transportista, fecha_entrega_cliente, fecha_estimada_entrega)

olist_detalle_pedidos: (id_pedido PK FK, id_item_pedido PK, id_producto FK, id_vendedor FK, fecha_limite_envio, precio, valor_flete)

olist_productos: (id_producto PK, categoria, peso_g, longitud_cm, altura_cm, ancho_cm)

olist_vendedores: (id_vendedor PK, codigo_postal FK)

olist_pagos_pedido: (id_pago PK, id_pedido FK, secuencia_pago, tipo_pago, cuotas_pago, valor_pago)

olist_valoraciones: (id_valoraciones PK, id_pedido FK, puntaje, mensaje_comentario, fecha_creacion, fecha_respuesta)

olist_geolocalizacion: (Codigo_postal PK, ciudad, estado)

3.5. Normalización (1FN-2FN-3FN)

Para garantizar la integridad y consistencia del modelo relacional, se aplicó el proceso de normalización en tres etapas progresivas. A continuación, se justifica el cumplimiento de cada forma normal sobre las tablas del dataset OLIST.

Primera Forma Normal (1FN)

La Primera Forma Normal establece que cada atributo de una tabla debe contener únicamente valores atómicos, es decir, un solo valor por campo, sin grupos repetidos, y que cada tabla debe contar con una llave primaria definida.

Al revisar cada tabla del dataset se verificó lo siguiente:

- **olist_clientes:** Cada registro representa a un único cliente con sus datos de identificación y ubicación. Atributos como id_unico_cliente, codigo_postal, ciudad y estado, entre otros, almacenan un solo valor por fila, sin combinar información de múltiples clientes en una misma celda. La llave primaria es id_cliente.
- **olist_pedidos:** Cada registro corresponde a una única orden de compra. Los atributos de fecha, como fecha_compra, fecha_aprobacion y fecha_estimada_entrega, entre otros, registran cada evento del ciclo de vida del pedido de forma independiente, sin agrupar múltiples valores en un solo campo. La llave primaria es id_pedido.
- **olist_detalle_pedidos:** Cada registro representa un único artículo dentro de un pedido. Atributos como precio, valor_flete, id_producto e id_vendedor, entre otros, contienen un único valor por fila. La llave primaria es compuesta: (id_pedido, id_item_pedido).

- **olist_productos:** Cada registro describe un único producto del catálogo. Sus atributos como categoria, peso_g, longitud_cm y altura_cm, entre otros, almacenan valores individuales por registro. La llave primaria es id_producto.
- **olist_vendedores:** Cada registro identifica a un único vendedor. Atributos como codigo_postal, ciudad y estado, entre otros, contienen un solo valor por fila. La llave primaria es id_vendedor.
- **olist_pagos_pedido:** Cada registro corresponde a una única transacción de pago. Atributos como tipo_pago, cuotas_pago y valor_pago, entre otros, contienen un solo valor por fila, lo que permite registrar múltiples métodos de pago para un mismo pedido como filas independientes. La llave primaria es id_pago.
- **olist_valoraciones:** Cada registro almacena la reseña de un cliente sobre un pedido específico. Atributos como puntaje, mensaje_comentario y fecha_creacion, entre otros, registran un único valor por reseña. La llave primaria es id_valoraciones.

Todas las tablas del modelo cumplen con la Primera Forma Normal desde el dataset original, por lo que no fue necesario realizar ninguna modificación en esta etapa.

Segunda Forma Normal (2FN)

La Segunda Forma Normal exige que, además de cumplir la 1FN, todos los atributos no clave dependan de la totalidad de la llave primaria y no de una parte de ella. Este tipo de dependencia, llamada dependencia parcial, solo puede ocurrir en tablas que tienen llave primaria compuesta, ya que en una llave simple no existen "partes" de las cuales depender parcialmente.

En el modelo, las tablas olist_clientes, olist_vendedores, olist_productos, olist_pedidos, olist_pagos_pedido, olist_valoraciones cuentan con llave primaria simple, por lo que la 2FN se cumple de forma directa en cada una de ellas sin necesidad de análisis adicional. La única tabla que requirió verificación es **olist_detalle_pedidos**, al ser la única con llave primaria compuesta: (id_pedido, id_item_pedido).

Atributo	¿Depende solo de id_pedido?	¿Depende solo de id_item_pedido?	¿Depende de la PK completa?
id_producto	No	No	Sí
id_vendedor	No	No	Sí
fecha_limite_envio	No	No	Sí
precio	No	No	Sí
valor_flete	No	No	Sí

Ningún atributo depende de una sola parte de la llave, todos dependen de la combinación completa (id_pedido, id_item_pedido). Por ejemplo, el precio de un ítem no corresponde

únicamente al pedido ni únicamente al número de ítem, sino a ambos en conjunto. El modelo cumple con la Segunda Forma Normal sin necesidad de modificaciones.

Tercera Forma Normal (3FN)

La Tercera Forma Normal establece que no deben existir dependencias transitivas, es decir, ningún atributo no clave debe depender de otro atributo no clave, sino únicamente de la llave primaria.

Al analizar las tablas del dataset se detectaron dependencias transitivas en dos tablas: olist_clientes y olist_vendedores

- Problema en olist_clientes:

La tabla original tenía la siguiente estructura:

id_cliente (PK)	id_unico_cliente	codigo_postal	ciudad	estado
abc123	xyz789	1310	São Paulo	SP
def456	uvw321	1310	São Paulo	SP
ghi789	rst654	20040	Rio de Janeiro	RJ

La dependencia transitiva detectada fue:

id_cliente → codigo_postal → ciudad, estado

Los atributos ciudad y estado no dependían directamente de id_cliente, sino de codigo_postal. Como se puede ver en la tabla, dos clientes distintos con el mismo código postal repetían los mismos valores de ciudad y estado, generando redundancia innecesaria.

- Problema en olist_vendedores:

La tabla original presentaba el mismo problema:

id_vendedor (PK)	codigo_postal	ciudad	estado
ven001	1310	São Paulo	SP
ven002	1310	São Paulo	SP
ven003	20040	Rio de Janeiro	RJ

La dependencia transitiva era idéntica:

id_vendedor → codigo_postal → ciudad, estado

Al igual que en olist_clientes, en la tabla olist_vendedores ciudad y estado dependen de codigo_postal y no de la llave primaria id_vendedor, lo que generaba la misma redundancia en la tabla.

Solución aplicada:

Para eliminar las dependencias transitivas en ambas tablas, se creó la tabla olist_geolocalizacion con codigo_postal como llave primaria, trasladando los atributos ciudad y estado a esta nueva

entidad. Tanto olist_clientes como olist_vendedores conservan únicamente codigo_postal como llave foránea que referencia a olist_geolocalizacion.

- **Tabla olist_geolocalizacion (nueva):**

codigo_postal (PK)	ciudad	estado
1310	São Paulo	SP
20040	Rio de Janeiro	RJ

Tabla olist_clientes (después de 3FN):

id_cliente (PK)	id_unico_cliente	codigo_postal (FK)
abc123	xyz789	1310
def456	uvw321	1310
ghi789	rst654	20040

-Tabla olist_vendedores (después de 3FN):

id_vendedor (PK)	codigo_postal (FK)
ven001	1310
ven002	1310
ven003	20040

Centralizar la ubicación en olist_geolocalizacion no solo evita la redundancia de texto, sino que optimiza el espacio en el disco del servidor y previene anomalías de actualización (por ejemplo, si una ciudad cambia de nombre, se actualiza en un solo registro y no en 100,000 pedidos).

Con esta modificación, ciudad y estado se almacenan una única vez por código postal, eliminando la redundancia y garantizando que cualquier actualización geográfica se realice en un solo lugar. El resto de las tablas no presentaron dependencias transitivas, por lo que no requirieron modificaciones adicionales.

Tras la creación de la tabla olist_geolocalizacion, el modelo queda conformado por 8 tablas completamente normalizadas hasta la Tercera Forma Normal.

3.6. Script DDL con restricciones de integridad

A continuación, el siguiente script DDL (Data Definition Language) presenta la definición completa de la base de datos, incluyendo la creación de las 8 tablas con sus respectivas llaves primarias, llaves foráneas y restricciones de integridad que garantizan la consistencia y validez de los datos almacenados.

-- BASE DE DATOS: OLIST E-Commerce

<pre> CREATE DATABASE OLIST_DB ON PRIMARY (name = N'OlistData', filename = N'C:\BD\OlistData.mdf', size = 50MB, maxsize = unlimited, filegrowth = 10%) LOG ON (name = N'OlistDataLog', filename = N'C:\BD\OlistDataLog.ldf', size = 10MB, maxsize = 500MB, filegrowth = 5MB); COLLATE Latin1_General_100_CI_AS_SC_UTF8 GO </pre>
-- NOS UBICAMOS EN LA BASE DE DATOS
<pre> USE OLIST_DB; GO </pre>
-- Tabla: GEOLOCALIZACION -- Creada como resultado de la 3FN para eliminar -- dependencias transitivas en CLIENTES y VENDEDORES
<pre> CREATE TABLE olist_geolocalizacion (codigo_postal VARCHAR(10) NOT NULL, ciudad VARCHAR(100) NOT NULL, estado CHAR(2) NOT NULL, CONSTRAINT PK_GEOLOCALIZACION PRIMARY KEY (codigo_postal)); GO </pre>
-- Tabla: CLIENTES -- Almacena la información de cada cliente por pedido
<pre> CREATE TABLE olist_clientes (id_cliente VARCHAR(50) NOT NULL, id_unico_cliente VARCHAR(50) NOT NULL, codigo_postal VARCHAR(10) NOT NULL, CONSTRAINT PK_CLIENTES PRIMARY KEY (id_cliente), CONSTRAINT FK_CLI_GEO FOREIGN KEY (codigo_postal) REFERENCES olist_geolocalizacion (codigo_postal), CONSTRAINT UQ_CLIENTE_UNICO UNIQUE (id_unico_cliente)); GO </pre>
-- Tabla: VENDEDORES -- Almacena información de los vendedores de la plataforma
<pre> CREATE TABLE olist_vendedores (id_vendedor VARCHAR(50) NOT NULL, codigo_postal VARCHAR(10) NOT NULL, CONSTRAINT PK_VENDEDORES PRIMARY KEY (id_vendedor), CONSTRAINT FK_VEN_GEO FOREIGN KEY (codigo_postal) </pre>

REFERENCES olist_geolocalizacion (codigo_postal)); GO
-- Tabla: PRODUCTOS -- Características físicas y de categoría de cada producto
CREATE TABLE olist_productos (id_producto VARCHAR(50) NOT NULL, categoria VARCHAR(100) NULL, peso_g INT NULL, longitud_cm INT NULL, altura_cm INT NULL, ancho_cm INT NULL, CONSTRAINT PK_PRODUCTOS PRIMARY KEY (id_producto), CONSTRAINT CHK_PESO CHECK (peso_g >= 0), CONSTRAINT CHK_LONGITUD CHECK (longitud_cm >= 0), CONSTRAINT CHK_ALTURA CHECK (altura_cm >= 0), CONSTRAINT CHK_ANCHO CHECK (ancho_cm >= 0)); GO
-- Tabla: PEDIDOS -- Registro de cada orden de compra y su ciclo de vida
CREATE TABLE olist_pedidos (id_pedido VARCHAR(50) NOT NULL, id_cliente VARCHAR(50) NOT NULL, estado_pedido VARCHAR(20) NOT NULL, fecha_compra DATETIME NOT NULL, fecha_aprobacion DATETIME NULL, fecha_entrega_transportista DATETIME NULL, fecha_entrega_cliente DATETIME NULL, fecha_estimada_entrega DATETIME NULL, CONSTRAINT PK_PEDIDOS PRIMARY KEY (id_pedido), CONSTRAINT FK_PED_CLI FOREIGN KEY (id_cliente) REFERENCES olist_clientes (id_cliente)); GO
-- Tabla: DETALLE_PEDIDOS -- Ítems individuales dentro de cada pedido -- Llave primaria compuesta: (id_pedido, id_item_pedido)

```

CREATE TABLE olist_detalle_pedidos (
  id_pedido      VARCHAR(50)  NOT NULL,
  id_item_pedido INT          NOT NULL,
  id_producto    VARCHAR(50)  NOT NULL,
  id_vendedor    VARCHAR(50)  NOT NULL,
  fecha_limite_envio DATETIME  NOT NULL,
  precio         DECIMAL(10,2) NOT NULL,
  valor_flete    DECIMAL(10,2) NOT NULL,
  CONSTRAINT PK_DETALLE PRIMARY KEY (id_pedido, id_item_pedido),
  CONSTRAINT FK_DET_PED FOREIGN KEY (id_pedido)
    REFERENCES olist_pedidos (id_pedido),
  CONSTRAINT FK_DET_PRO FOREIGN KEY (id_producto)
    REFERENCES olist_productos (id_producto),
  CONSTRAINT FK_DET_VEN FOREIGN KEY (id_vendedor)
    REFERENCES olist_vendedores (id_vendedor),
  CONSTRAINT CHK_PRECIO CHECK (precio >= 0),
  CONSTRAINT CHK_FLETE CHECK (valor_flete >= 0)
);
GO

```

-- Tabla: PAGOS

-- Métodos y montos de pago por pedido

-- Un pedido puede tener múltiples registros de pago

```

CREATE TABLE olist_pagos_pedido (
  id_pago      VARCHAR(60)  NOT NULL,
  id_pedido    VARCHAR(50)  NOT NULL,
  secuencia_pago INT        NOT NULL,
  tipo_pago    VARCHAR(50)  NOT NULL,
  cuotas_pago  INT          NOT NULL,
  valor_pago   DECIMAL(10,2) NOT NULL,
  CONSTRAINT PK_PAGOS PRIMARY KEY (id_pago),
  CONSTRAINT FK_PAG_PED FOREIGN KEY (id_pedido)
    REFERENCES olist_pedidos (id_pedido),
  CONSTRAINT CHK_CUOTAS CHECK (cuotas_pago >= 0),
  CONSTRAINT CHK_VALOR_PAGO CHECK (valor_pago >= 0)
);
GO

```

-- Tabla: VALORACIONES

-- Reseñas y calificaciones de clientes por pedido

```

CREATE TABLE olist_valoraciones (
  id_valoraciones VARCHAR(100) NOT NULL,
  id_pedido        VARCHAR(50) NOT NULL,
  puntaje          INT          NOT NULL,
  mensaje_comentario TEXT       NULL,
  fecha_creacion   DATETIME     NOT NULL,
  fecha_respuesta  DATETIME     NULL,
  CONSTRAINT PK_VALORACIONES PRIMARY KEY (id_valoraciones),
  CONSTRAINT FK_VAL_PED FOREIGN KEY (id_pedido)
    REFERENCES olist_pedidos (id_pedido),
  CONSTRAINT CHK_PUNTAJE CHECK (puntaje BETWEEN 1 AND 5)
);
GO

```

```
);  
GO
```

-- **ÍNDICES:** Se implementaron índices sobre las llaves foráneas y las columnas de fechas porque, tras analizar las preguntas de negocio de la Etapa 1, se determinó que estos campos serán los más consultados en las cláusulas JOIN y WHERE, mejorando así los tiempos de respuesta del motor SQL.

```
CREATE INDEX IDX_PED_CLIENTE ON olist_pedidos(id_cliente);  
CREATE INDEX IDX_DET_PRODUCTO ON  
olist_detalle_pedidos(id_producto);  
CREATE INDEX IDX_DET_VENDEDOR ON  
olist_detalle_pedidos(id_vendedor);  
CREATE INDEX IDX_PAG_PEDIDO ON olist_pagos_pedido(id_pedido);  
CREATE INDEX IDX_VAL_PEDIDO ON olist_valoraciones(id_pedido);  
CREATE INDEX IDX_PED_ESTADO ON olist_pedidos(estado_pedido);  
CREATE INDEX IDX_PED_FECHA ON olist_pedidos(fecha_compra);  
GO
```

4. ETAPA 3 - Implementación y Carga de Datos / ETL

4.1. Creación de la base de datos en Microsoft SQL Server

En este caso estamos usando el SQL Server

```
CREATE DATABASE OLIST_DB  
ON PRIMARY (  
    name = N'OlistData',  
    filename = N'C:\BD\OlistData.mdf',  
    size = 50MB,  
    maxsize = unlimited,  
    filegrowth = 10%  
)  
LOG ON (  
    name = N'OlistDataLog',  
    filename = N'C:\BD\OlistDataLog.ldf',  
    size = 10MB,  
    maxsize = 500MB,  
    filegrowth = 5MB  
)  
COLLATE Latin1_General_100_CI_AS_SC_UTF8;  
GO  
USE OLIST_DB;  
GO
```

4.2. Ejecución del script DDL y verificar la creación correcta de tablas

Para evitar errores durante el proceso de BULK INSERT debido a los distintos formatos de fecha del archivo CSV original, las tablas se crearon inicialmente como Tablas Temporales utilizando el tipo de dato VARCHAR para las fechas. Posteriormente, como se evidencia, estos datos fueron transformados y convertidos a tipos DATETIME estrictos en las tablas definitivas.

Tabla olist_geolocalizacion

```

CREATE TABLE olist_geolocalizacion (
    codigo_postal VARCHAR(50) NOT NULL,
    ciudad VARCHAR(150) NOT NULL,
    estado VARCHAR(10) NOT NULL,
    CONSTRAINT PK_GEOLOCALIZACION PRIMARY KEY (codigo_postal)
);
GO

```

Tabla olist_clientes

```

CREATE TABLE olist_clientes (
    id_cliente VARCHAR(50) NOT NULL,
    id_unico_cliente VARCHAR(50) NOT NULL,
    codigo_postal VARCHAR(50) NOT NULL,
    CONSTRAINT PK_CLIENTES PRIMARY KEY (id_cliente),
    CONSTRAINT FK_CLI_GEO FOREIGN KEY (codigo_postal)
        REFERENCES olist_geolocalizacion (codigo_postal),
    CONSTRAINT UQ_CLIENTE_UNICO UNIQUE (id_unico_cliente)
);
GO

```

Tabla olist_vendedores

```

CREATE TABLE olist_vendedores (
    id_vendedor VARCHAR(50) NOT NULL,
    codigo_postal VARCHAR(50) NOT NULL,
    CONSTRAINT PK_VENDEDORES PRIMARY KEY (id_vendedor),
    CONSTRAINT FK_VEN_GEO FOREIGN KEY (codigo_postal)
        REFERENCES olist_geolocalizacion (codigo_postal)
);
GO

```

Tabla olist_productos

```

CREATE TABLE olist_productos (
    id_producto VARCHAR(50) NOT NULL,
    categoria NVARCHAR(100) NULL,
    peso_g INT NULL,
    longitud_cm INT NULL,
    altura_cm INT NULL,
    ancho_cm INT NULL,
    CONSTRAINT PK_PRODUCTOS PRIMARY KEY (id_producto),
    CONSTRAINT CHK_PESO CHECK (peso_g >= 0),
    CONSTRAINT CHK_LONGITUD CHECK (longitud_cm >= 0),
    CONSTRAINT CHK_ALTURA CHECK (altura_cm >= 0),
    CONSTRAINT CHK_ANCHO CHECK (ancho_cm >= 0)
);
GO

```

Tabla olist_pedidos

```

CREATE TABLE olist_pedidos (
    id_pedido VARCHAR(50) NOT NULL,
    id_cliente VARCHAR(50) NOT NULL,
    estado_pedido VARCHAR(50) NOT NULL,
    fecha_compra VARCHAR(50) NOT NULL, --En VARCHAR PORQUE ES
    (DD/MM/AA HH:MM)
    fecha_aprobacion VARCHAR(50) NULL, --En VARCHAR PORQUE ES
    (DD/MM/AA HH:MM)
    fecha_entrega_transportista VARCHAR(50) NULL, --En VARCHAR PORQUE ES
    (DD/MM/AA HH:MM)
    fecha_entrega_cliente VARCHAR(50) NULL, --En VARCHAR PORQUE ES
    (DD/MM/AA HH:MM)
    fecha_estimada_entrega VARCHAR(50) NULL, --En VARCHAR PORQUE ES
    (DD/MM/AA HH:MM)
    CONSTRAINT PK_PEDIDOS PRIMARY KEY (id_pedido),
    CONSTRAINT FK_PED_CLI FOREIGN KEY (id_cliente)
        REFERENCES olist_clientes (id_cliente)
);
GO

```

Tabla olist_detalle_pedidos

```

CREATE TABLE olist_detalle_pedidos (
  id_pedido          VARCHAR(50)      NOT NULL,
  id_item_pedido     INT              NOT NULL,
  id_producto        VARCHAR(50)      NOT NULL,
  id_vendedor        VARCHAR(50)      NOT NULL,
  fecha_limite_envio VARCHAR(100)      NOT NULL, --En VARCHAR PORQUE ES (DD/MM/AA
HH:MM)
  precio             DECIMAL(10,2)    NOT NULL,
  valor_flete        DECIMAL(10,2)    NOT NULL,
  CONSTRAINT PK_DETALLE PRIMARY KEY (id_pedido, id_item_pedido),
  CONSTRAINT FK_DET_PED FOREIGN KEY (id_pedido)
    REFERENCES olist_pedidos (id_pedido),
  CONSTRAINT FK_DET_PRO FOREIGN KEY (id_producto)
    REFERENCES olist_productos (id_producto),
  CONSTRAINT FK_DET_VEN FOREIGN KEY (id_vendedor)
    REFERENCES olist_vendedores (id_vendedor),
  CONSTRAINT CHK_PRECIO CHECK (precio >= 0),
  CONSTRAINT CHK_FLETE CHECK (valor_flete >= 0)
);
GO

```

Tabla olist pagos pedido

```

CREATE TABLE olist_pagos_pedido (
  id_pago          VARCHAR(60)      NOT NULL,
  id_pedido        VARCHAR(50)      NOT NULL,
  secuencia_pago   INT              NOT NULL,
  tipo_pago        VARCHAR(50)      NOT NULL,
  cuotas_pago      INT              NOT NULL,
  valor_pago       DECIMAL(10,2)    NOT NULL,

  CONSTRAINT PK_PAGOS PRIMARY KEY (id_pago),

  CONSTRAINT FK_PAG_PED FOREIGN KEY (id_pedido)
    REFERENCES olist_pedidos (id_pedido),

  CONSTRAINT CHK_CUOTAS CHECK (cuotas_pago >= 0),
  CONSTRAINT CHK_VALOR_PAGO CHECK (valor_pago >= 0)
);
GO

```

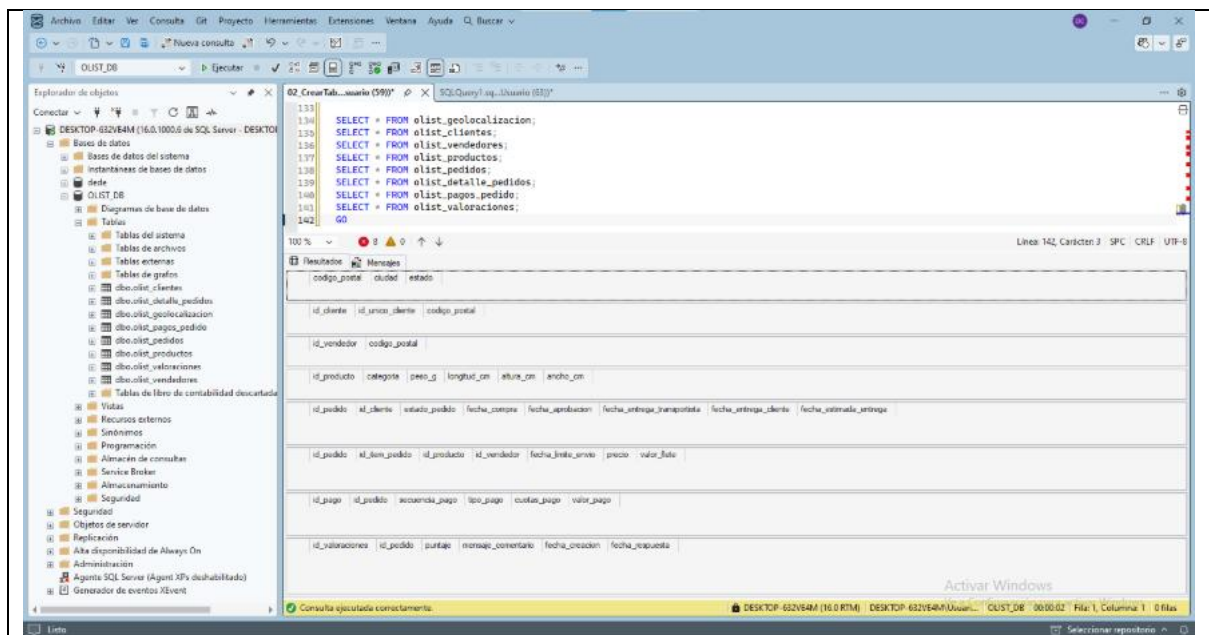
Tabla Valoraciones

```

CREATE TABLE olist_valoraciones (
  id_valoraciones   VARCHAR(100)     NOT NULL,
  id_pedido         VARCHAR(50)      NOT NULL,
  puntaje           INT              NOT NULL,
  mensaje_comentario NVARCHAR(MAX)    NULL,      --NVARCHAR(MAX) para aguantar
todas las tildes y caracteres
  fecha_creacion    VARCHAR(100)     NOT NULL, --En VARCHAR PORQUE ES (DD/MM/AA
HH:MM)
  fecha_respuesta   VARCHAR(100)     NULL,      --En VARCHAR PORQUE ES (DD/MM/AA
HH:MM)
  CONSTRAINT PK_VALORACIONES PRIMARY KEY (id_valoraciones),
  CONSTRAINT FK_VAL_PED FOREIGN KEY (id_pedido)
    REFERENCES olist_pedidos (id_pedido)
);
GO

```

COMPROBACION



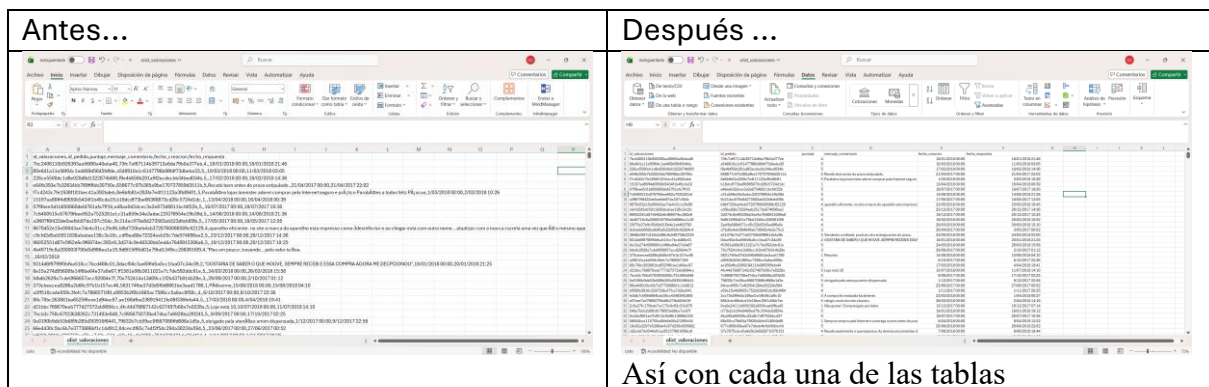
4.3. Implementación del proceso ETL:

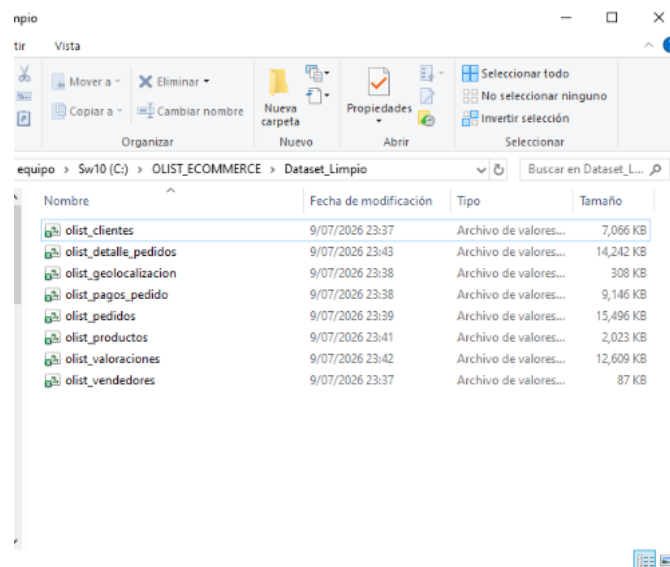
4.3.1. Extracción: lectura del archivo CSV/XLSX del dataset

Para el desarrollo del proyecto se empleará el dataset OLIST E-Commerce Public Dataset, La fuente de descarga fue a través de Kaggle y los datos se encontraban en archivos CSV. Cada archivo representaba una entidad diferente del sistema de comercio electrónico.

Crear un libro nuevo -> Datos -> De texto/csv -> Seleccionar csv -> Delimitado [coma] -> Origen archivo [65001 unicode (UTF-8)]

Esto con el fin de que se separen todos los datos





4.3.2. Transformación y limpieza de datos

- ❖ Tratamiento de Nulos: Durante la revisión de los archivos en Power Query, se determinó que los campos vacíos en las columnas de comentarios (mensaje_comentario) y fechas de respuesta opcionales representan eventos naturales del negocio (clientes que deciden no escribir una reseña). Por ende, se configuró el proceso para preservar estos espacios como valores NULL lógicos, evitando la pérdida de registros de calificación (puntuajes).
- ❖ Normalización de Formatos (fechas): Las variables cronológicas del dataset original presentaban una estructura combinada de fecha y hora en formato de texto plano. En Power Query se normalizó su estructura a un formato de texto unificado y consistente (AAAA-MM-DD HH:MM:SS), de manera que, al ser cargadas inicialmente como texto en SQL Server, pudieran reconocerse sin ambigüedad y convertirse posteriormente a tipos de datos temporales sin pérdida de granularidad.
- ❖ Codificación de Caracteres (Internacionalización): Dado que el dataset proviene de un ecosistema de comercio electrónico brasileño, los campos de texto libre contienen caracteres especiales (ej. ç, ã, ó). Durante la exportación desde Power Query se mantuvo la codificación UTF-8 (CodePage 65001), garantizando que estos caracteres se conservaran correctamente al momento de la carga en SQL Server.

- ❖ Codificación de Caracteres (Internacionalización): Dado que el dataset proviene de un ecosistema de comercio electrónico brasileño, los campos de texto libre contienen caracteres especiales (ej. ç, ã, ó). La transformación crítica consistió en aplicar la codificación UTF-8 (CodePage 65001) y asignar propiedades de almacenamiento NVARCHAR, previniendo errores de incompatibilidad (type mismatch) y garantizando la lectura exacta de las opiniones.

Se altero la columna mensaje_comentario para que acepte caarakteres especiales
 -- Evidencia de la configuración aplicada en la base de datos:
 ALTER TABLE VALORACIONES ALTER COLUMN mensaje_comentario NVARCHAR(MAX) NULL;
 -- Aplicado en el BULK INSERT mediante: CODEPAGE = '65001'

- ❖ Durante la validación del proceso de carga se identificó que el atributo **id_unico_cliente** presenta valores repetidos, ya que representa al mismo cliente asociado a distintos registros (id_cliente). Por ello se eliminó la restricción UNIQUE definida inicialmente, manteniendo id_cliente como clave primaria y preservando la integridad de los datos.

```
ALTER TABLE olist_clientes
DROP CONSTRAINT UQ_CLIENTE_UNICO;
GO
```

4.3.3. Carga: inserción de datos en la base de datos.

Tabla Geolocalización

```
--Comprobación (visualizar los primeros 50 datos)
SELECT TOP 50 * FROM olist_geolocalizacion;
```

Tabla Clientes

1. ANTES DE INSERTAR LOS DATOS, ALTERAMOS LA TABLA PARA QUE NO HAYA PROBLEMA CON LOS ID DUPLICADOS

```
--CARGAR LOS DATOS DE LA TABLA CLIENTES
BULK INSERT olist_clientes
FROM 'C:\OLIST_ECOMMERCE\olist_clientes.csv'
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '0x0a',
    CODEPAGE = '65001'
);
GO
```

--COMPROBACION DEL TOTAL DE CLIENTES

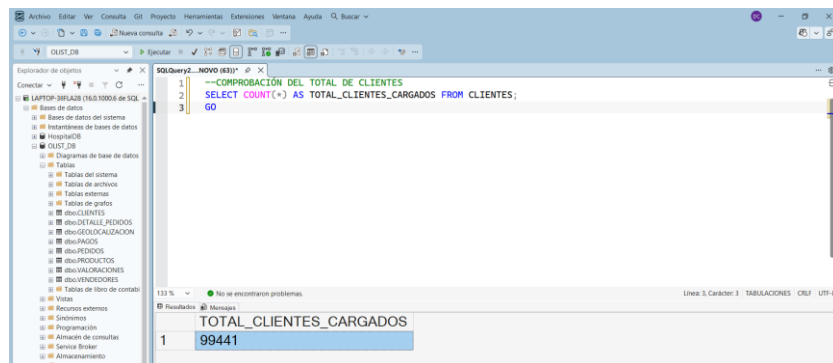
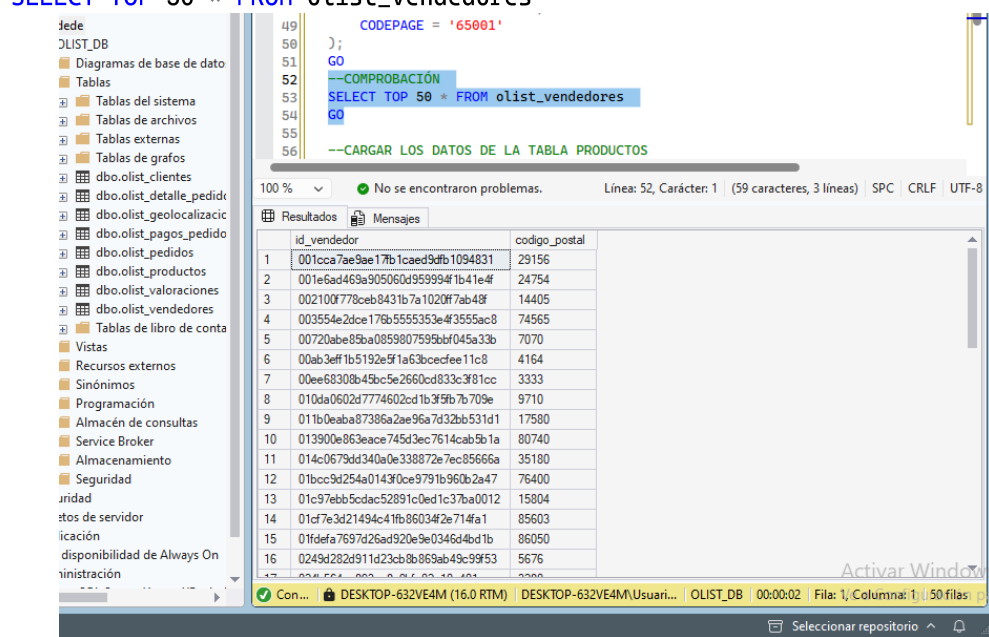


Tabla Vendedores

```
--CARGAR LOS DATOS DE LA TABLA VENDEDORES
BULK INSERT olist_vendedores
FROM 'C:\OLIST_ECOMMERCE\Dataset_Limpio\olist_vendedores.csv'
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '0x0a',
    CODEPAGE = '65001'
);
GO
```

```
--COMPROBACION
SELECT TOP 50 * FROM olist_vendedores
```



id_vendedor	codigo_postal
001cca7ae9ae17b1caed9dfb1094831	29156
001e6ad469a905060d959994f1b41e4f	24754
002100f778ceb8431b7a1020f7ab48f	14405
003554e2dce17b5555353e4f3555ac8	74565
00720abe85ba0859807595b6f045a33b	7070
00ab3eff1b5192e9f1a63bcecf11c8	4164
00ee68308b45bc5e2660cd83c3f81cc	3333
010da0602d7774602cd1b3f9b7b709e	9710
011b0eaba87386a2ae96a7d32bb531d1	17580
013900e863eace745d3ec7614cab5b1a	80740
014c0679dd340a0e338872e7ec85666a	35180
01bcc9d254a0143f0ce979b1b960b2a47	76400
01c97ebb5cdac52891cded1c37ba0012	15804
01cf7e3d21494c41fb86034f2e714fa1	85603
01fdefa7697d26ad920e9e0346d4bd1b	86050
0249d282d911d23cb8b869ab49c99f53	5676

Tabla Productos

```
--CARGAR LOS DATOS DE LA TABLA PRODUCTOS
BULK INSERT olist_productos
FROM 'C:\OLIST_ECOMMERCE\Dataset_Limpio\olist_productos.csv'
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '0x0a',
    CODEPAGE = '65001'
);
GO
```

```
--COMPROBACION
SELECT TOP 50 * FROM olist_productos;
GO
```

The screenshot shows the SQL Server Enterprise Manager interface. On the left, the 'Explorador de objetos' (Object Explorer) displays the 'OLIST_DB' database structure, including tables like 'olist_clientes', 'olist_detalle_pedido', 'olist_geolocalizaciones', 'olist_pagos_pedido', 'olist_pedidos', 'olist_productos', 'olist_valoraciones', and 'olist_vendedores'. The main window shows a SQL query window titled 'SQLQuery1.s...suario (63)' with the following code:

```
--CARGAR LOS DATOS DE LA TABLA PRODUCTOS
BULK INSERT olist_productos
FROM 'C:\OLIST_ECOMMERCE\Dataset_Limpio\olist_productos.csv'
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '0x0a',
    CODEPAGE = '65001'
);
GO
--COMPROBACIÓN
SELECT TOP 50 * FROM olist_productos;
```

The query results are displayed in a table with the following columns: id_producto, categoria, peso_g, longitud_cm, altura_cm, and ancho_cm. The results show 16 rows of data, including products like 'perfumaria', 'automotivo', 'cama_mesa_banho', 'utilidades_domesticas', 'relojes_presentes', 'automotivo', 'cool_stuff', 'consoles_games', 'cama_mesa_banho', 'moveis_decoracao', 'cama_mesa_banho', 'beleza_saude', 'fashion_calçados', 'informatica_acessorios', 'utilidades_domesticas', and 'cool_stuff'.

Tabla Pedidos

```
--CARGAR LOS DATOS DE LA TABLA PEDIDOS
```

```
BULK INSERT olist_pedidos
FROM 'C:\OLIST_ECOMMERCE\Dataset_Limpio\olist_pedidos.csv'
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '0x0a',
    CODEPAGE = '65001'
);
GO
```

```
--COMPROBACION
```

```
SELECT TOP 50 * FROM olist_pedidos;
GO
```

Archivo Editar Ver Consulta Git Proyecto Herramientas Extensiones Ventana Ayuda Q Buscar

OLIST_DB Ejecutar

Explorador de objetos

Conectar

DESKTOP-632VE4M (16.0.1000.6 de SQ

Bases de datos

Instantáneas de bases de datos

dede

OLIST_DB

Diagramas de base de dato

Tablas

Tablas del sistema

Tablas de archivos

Tablas externas

Tablas de grafos

dbo.olist_clientes

dbo.olist_detalle_pedido

dbo.olist_geolocalizac

dbo.olist_pagos_pedido

dbo.olist_pedidos

dbo.olist_productos

dbo.olist_valoraciones

dbo.olist_vendedores

Tablas de libro de conta

Vistas

Recursos externos

Sinónimos

Programación

Almacén de consultas

Service Broker

Almacenamiento

Seguridad

Objetos de servidor

Replicación

Alta disponibilidad de Always On

Administración

SQLQuery1.s...uario (63))

02_CrearTabla...Usuario (59))

```

70 --CARGAR LOS DATOS DE LA TABLA PEDIDOS
71
72 BULK INSERT olist_pedidos
73 FROM 'C:\OLIST_ECOMMERCE\Dataset_Limpio\olist_pedidos.csv'
74 WITH (
75     FIRSTROW = 2,
76     FIELDTERMINATOR = ',',
77     ROWTERMINATOR = '0x0a',
78     CODEPAGE = '65001'
79 );
80 GO
81 --COMPROBACIÓN
82 SELECT TOP 50 * FROM olist_pedidos;
83 GO

```

100 % No se encontraron problemas. Línea: 81, Carácter: 16 SPC CRLF UTF-8

Resultados Mensajes

	id_pedido	id_cliente	estado_pedido	fecha_compra	fec
1	00010242e8c5a6d1ba2dd792cb16214	3ce436f183e68e07877b285a838db11a	delivered	2017-09-13 08:59:00	20
2	00018f772f0320c557190d7a144bdd3	f6dd3ec061db4e3987629fe6b26e5c0ce	delivered	2017-04-26 10:53:00	20
3	000229ec398224ef6ca0657da4fc703e	6489ae5e433f3693df5ad4372dab6d3	delivered	2018-01-14 14:33:00	20
4	00024acbcd0a6daa1e931b038114c75	d4eb9395c8c0431ee92fce09860c5a06	delivered	2018-08-08 10:00:00	20
5	00042b26cf59d7ce69dfabb4e55b4fd9	58dbd0b2d70206bf40e62cd34e84d795	delivered	2017-02-04 13:57:00	20
6	00048cc3ae777c65dbb7d2a0634bc1ea	816cbea969fe5b689b39cfc97a506742	delivered	2017-05-15 21:42:00	20
7	00054e8431b9d7675808cb819b4a32	32e2e6ab09e778d99bf2e0ecd4898718	delivered	2017-12-10 11:53:00	20
8	000579fe39319847cbb9d288c5617fa6	9ed5e522dd9dd85b4af4a077526d8117	delivered	2018-07-04 12:08:00	20
9	0005a1a1728c9d785b8e2b08e904576c	16150771dfd4776261284213b89c304e	delivered	2018-03-19 18:40:00	20
10	0009f50442cb953dcd1d21e1fb923495	351d3cb2cee3c7fd0af6616c82df21d3	delivered	2018-07-02 13:59:00	20
11	00061f2a7bc09da83e415a52dc8a4af1	c6fc061d8fab1e2b2eac259bac71a49	delivered	2018-03-24 22:16:00	20
12	00063b381e2406b52ad429470734ebd5	6a899e55865de6549a58d2c6845e5604	delivered	2018-07-27 17:21:00	20
13	0006ec9db01a64e59a68b2c340bf65a7	5d178120c29c61748ea95bac23cb8f25	delivered	2018-07-24 17:04:00	20
14	0008288aa423d2a3f00fcb17cd7d8719	2355af7c75e7c98b43a87b2a7210dc5	delivered	2018-02-13 22:10:00	20
15	0009792311464db532ff765bf7b182ae	2a30c97668e81df7c17a8b14447aeeba	delivered	2018-08-14 20:43:00	20
16	000a0a017301c70c47170a107c54c12	0c7cfbda4fbac5c29afba04f16c7b	delivered	2018-04-26 06:10:00	20

Activar Windows

Con... DESKTOP-632VE4M (16.0 RTM) DESKTOP-632VE4M\Usuari... OLIST_DB 00:00:00 Fila: 1, Columna: 1 150 filas

Listo Seleccionar repositorio

Tabla Detalle de Pedidos

--CARGAR LOS DATOS DE LA TABLA DETALLE _PEDIDOS

```

BULK INSERT olist_detalle_pedidos
FROM 'C:\OLIST_ECOMMERCE\Dataset_Limpio\olist_detalle_pedidos.csv'
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '0x0a',
    CODEPAGE = '65001'
);
GO

```

-- COMPROBACION

```

SELECT TOP 50 * FROM olist_detalle_pedidos;
GO

```

Archivo Editar Ver Consulta Git Proyecto Herramientas Extensiones Ventana Ayuda Q Buscar

OLIST_DB Ejecutar

Explorador de objetos

Conectar

DESKTOP-632VE4M (16.0.1000.6 de SQ

Bases de datos

Bases de datos del sistema

Instantáneas de bases de datos

dede

OLIST_DB

Diagramas de base de dato

Tablas

Tablas del sistema

Tablas de archivos

Tablas externas

Tablas de grafos

dbo.olist_clientes

dbo.olist_detalle_pedidos

dbo.olist_geolocalizac

dbo.olist_pagos_pedido

dbo.olist_pedidos

dbo.olist_productos

dbo.olist_valoraciones

dbo.olist_vendedores

Tablas de libro de conta

Vistas

Recursos externos

Sinónimos

Programación

Almacén de consultas

Service Broker

Almacenamiento

Seguridad

Seguridad

Objetos de servidor

Replicación

Alta disponibilidad de Always On

Administración

SQLQuery1.s...uario (63)*

02_CrearTabla...Usuario (59)

```

85 --CARGAR LOS DATOS DE LA TABLA DETALLE _PEDIDOS
86
87 BULK INSERT olist_detalle_pedidos
88 FROM 'C:\OLIST_ECOMMERCE\Dataset_Limpio\olist_detalle_pedidos.csv'
89 WITH (
90     FIRSTROW = 2,
91     FIELDTERMINATOR = ',',
92     ROWTERMINATOR = '0x0a',
93     CODEPAGE = '65001'
94 );
95 GO
96 -- COMPROBACIÓN
97 SELECT TOP 50 * FROM olist_detalle_pedidos;
98 GO

```

100 % 1 0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32 33 34 35 36 37 38 39 40 41 42 43 44 45 46 47 48 49 50 51 52 53 54 55 56 57 58 59 60 61 62 63 64 65 66 67 68 69 70 71 72 73 74 75 76 77 78 79 80 81 82 83 84 85 86 87 88 89 90 91 92 93 94 95 96 97 98

Linea: 95, Carácter: 4 SPC CRLF UTF-8

Resultados Mensajes

	id_pedido	id_item_pedido	id_producto	id_vendedor
1	00010242fe8c5a6d1ba2dd792cb16214	1	4244733e06e7ecb4970a6e2683c13e61	48436dade18ac8b2bce085
2	00018f772f0320c557190d7a144bdd3	1	e9f2d52b802189ee658865ca93d83a8f	dd7ddc04e1b6c2c614352b
3	000229ec398224ef6ca0657da4fc703e	1	c777355d18b72b67abbeef9d44df0fd	5b51032eddd24adc84c3f
4	00024acbcd0a6daa1e931b038114c75	1	7634da152a4610f1595efa32f14722fc	9d7a1d34a505240900642f
5	00042b26cf59d7ce69fabb4e55b4fd9	1	ac6c3623068f30de03045865e4e10089	df560393f3a51e74553ab94
6	00048cc3ae777c65dbb7d2a0634bc1ea	1	ef92defde845ab84509d70c526ef70f	6426d21aca402a131fc0a5
7	00054e8431b9d7675808cb819b4a32	1	8d4f2bb7e93e6710a28f34fa83ee7d28	7040e82f899a04d1b434b7
8	000579fe39319847cbb9d288c5617fa6	1	557d850972a7d9f792fd18ae1400d9b6	5996cddab893a4652a155f
9	0005a1a1728c9d785b8e2b08b904576c	1	310ae3c140f94b03219ad0adc3c778f	a416b6a846a1172439302f
10	0005f50442cb953dcd1d21e1fb923495	1	4535b0e1091c278df193e5a1d63b39f	ba143b09f0110f0dc71ad7
11	00061f2a7bc09da83e415a52dc8a4af1	1	d63c1011f49d98b976c35295b1c4bea	cc419e0650a3c5ba77189e
12	00063b381e2406b52ad429470734ebd5	1	f177554ea93259a5b282f24e33f65ab6	8602a61d680a10a82ccee
13	0006ec9db01a64e59a68b2c340bf65a7	1	99a4788cb24856965c36a24e339b6058	4a3ca9315b744ce9f9e937
14	0008288aa423d2a3f00cb17cd7d8719	1	368c6c730842d78016ad823897a372db	1f50f920176fa81db9949f
15	0008288aa423d2a3f00cb17cd7d8719	2	368c6c730842d78016ad823897a372db	1f50f920176fa81db9949f
16	00007071114c4d4c274f7c5f7a107...	1	0...	...

Activar Windows

Con... DESKTOP-632VE4M (16.0 RTM) DESKTOP-632VE4M\Usuari... OLIST_DB 00:00:00 Fila: 1, Columna: 1 150 filas

Listo

Seleccionar repositorio

Tabla Pagos

--CARGAR LOS DATOS DE LA TABLA PAGOS

```

BULK INSERT olist_pagos_pedido
FROM 'C:\OLIST_ECOMMERCE\Dataset_Limpio\olist_pagos_pedido.csv'
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ',',
    ROWTERMINATOR = '0x0a',
    CODEPAGE = '65001'
);
GO

```

```

-- COMPROBACION
SELECT TOP 50 * FROM olist_pagos_pedido;
GO

```

Archivo Editar Ver Consulta Git Proyecto Herramientas Extensiones Ventana Ayuda Q Buscar

OLIST_DB Ejecutar

Explorador de objetos

Conectar

DESKTOP-632VE4M (16.0.1000.6 de SQ

Bases de datos

Instantáneas de bases de datos

dede

OLIST_DB

Diagramas de base de dato

Tablas

Tablas del sistema

Tablas de archivos

Tablas externas

Tablas de grafos

dbo.olist_clientes

dbo.olist_detalle_pedido

dbo.olist_geolocalizac

dbo.olist_pagos_pedido

dbo.olist_pedidos

dbo.olist_productos

dbo.olist_valoraciones

dbo.olist_vendedores

Tablas de libro de conta

Vistas

Recursos externos

Sinónimos

Programación

Almacén de consultas

Service Broker

Almacenamiento

Seguridad

Seguridad

Objetos de servidor

Replicación

Alta disponibilidad de Always On

Administración

02_CrearTabla...Usuario (59)

SQLQuery1.s...uario (63)

```

101 --CARGAR LOS DATOS DE LA TABLA PAGOS
102 BULK INSERT olist_pagos_pedido
103 FROM 'C:\OLIST_ECOMMERCE\Dataset_Limpio\olist_pagos_pedido.csv'
104 WITH (
105     FIRSTROW = 2,
106     FIELDTERMINATOR = ',',
107     ROWTERMINATOR = '0x0a',
108     CODEPAGE = '65001'
109 );
110 GO
111 -- COMPROBACIÓN
112 SELECT TOP 50 * FROM olist_pagos_pedido;
113 GO
114
115

```

100 % No se encontraron problemas. Línea: 115, Carácter: 1 SPC CRLF UTF-8

Resultados Mensajes

	id_pago	id_pedido	secuencia_pago	tipo_pago	cuot.
1	00010242fe8c5a6d1ba2dd792cb16214-1	00010242fe8c5a6d1ba2dd792cb16214	1	credit_card	2
2	000187772f0320c557190d7a144bdd3-1	000187772f0320c557190d7a144bdd3	1	credit_card	3
3	000229ec398224ef6ca0657da4fc703e-1	000229ec398224ef6ca0657da4fc703e	1	credit_card	5
4	00024acbcd0fa6daa1e931b038114c75-1	00024acbcd0fa6daa1e931b038114c75	1	credit_card	2
5	00042b26cf59d7ce69dfabb4e55b4fd9-1	00042b26cf59d7ce69dfabb4e55b4fd9	1	credit_card	3
6	00048cc3ae777c65ddb7d2a0634bc1ea-1	00048cc3ae777c65ddb7d2a0634bc1ea	1	boleto	1
7	00054e8431b9d7675808bcb819fb4a32-1	00054e8431b9d7675808bcb819fb4a32	1	credit_card	1
8	000578fe39319847cbb9d288c5617fa6-1	000578fe39319847cbb9d288c5617fa6	1	credit_card	10
9	0005a1a1728c9d785b9e2b08b904576c-1	0005a1a1728c9d785b9e2b08b904576c	1	credit_card	3
10	000f50442cb953dcd1d21e1fb923495-1	000f50442cb953dcd1d21e1fb923495	1	credit_card	1
11	00061f2a7bc09da83e415a52dc8a4af1-1	00061f2a7bc09da83e415a52dc8a4af1	1	credit_card	3
12	00063b381e2406b52ad429470734ebd5-1	00063b381e2406b52ad429470734ebd5	1	credit_card	5
13	0006ec9db01a64e59a68b2c340bf65a7-1	0006ec9db01a64e59a68b2c340bf65a7	1	credit_card	4
14	0008288aa423d2a3f00fcb17cd7d8719-1	0008288aa423d2a3f00fcb17cd7d8719	1	boleto	1
15	0009792311464d4532f765f7b182ae-1	0009792311464d4532f765f7b182ae	1	boleto	1

Activar Windows

DESKTOP-632VE4M (16.0 RTM) | DESKTOP-632VE4M\Usuari... | OLIST_DB | 00:00:00 | Fila: 1, Columna: 1 | 50 filas

Elementos guardados

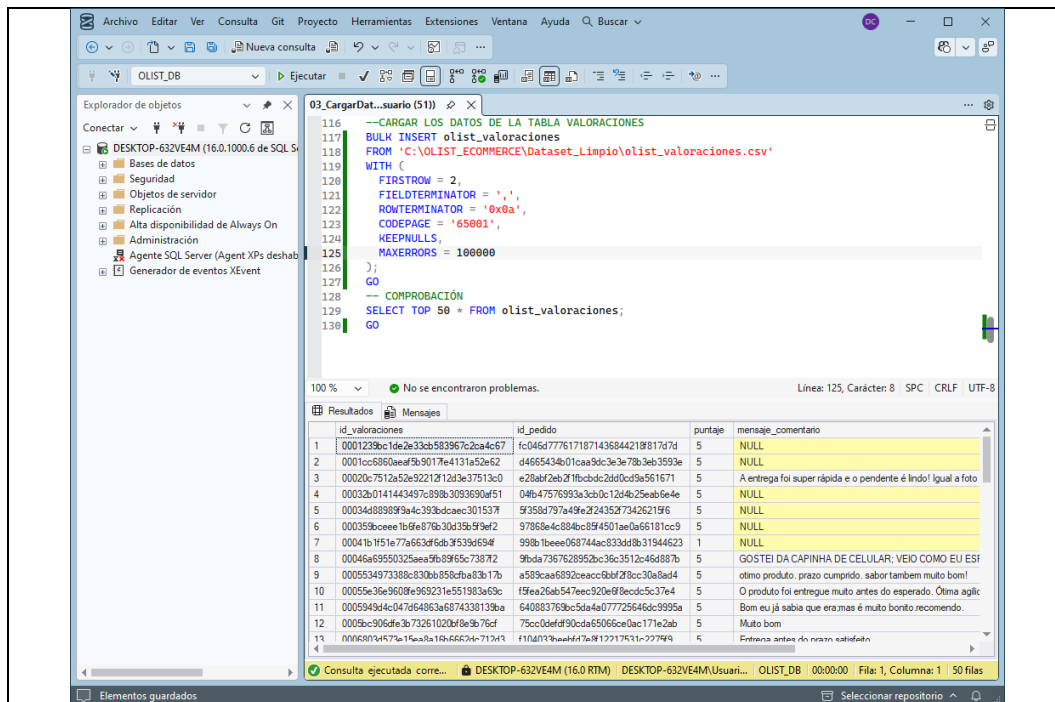
Seleccionar repositorio

Tabla Valoraciones

```

--CARGAR LOS DATOS DE LA TABLA VALORACIONES
BULK INSERT olist_valoraciones
FROM 'C:\OLIST_ECOMMERCE\Dataset_Limpio\olist_valoraciones.csv'
WITH (
    FIRSTROW = 2,
    FIELDTERMINATOR = ',', --
    ROWTERMINATOR = '0x0a',
    CODEPAGE = '65001',
    KEEPNULLS, -- Respeta las celdas vacías
    MAXERRORS = 100000 -- Salta cualquier registro corrupto en el
fondo
);
-- COMPROBACION
SELECT TOP 50 * FROM olist_valoraciones;
GO

```



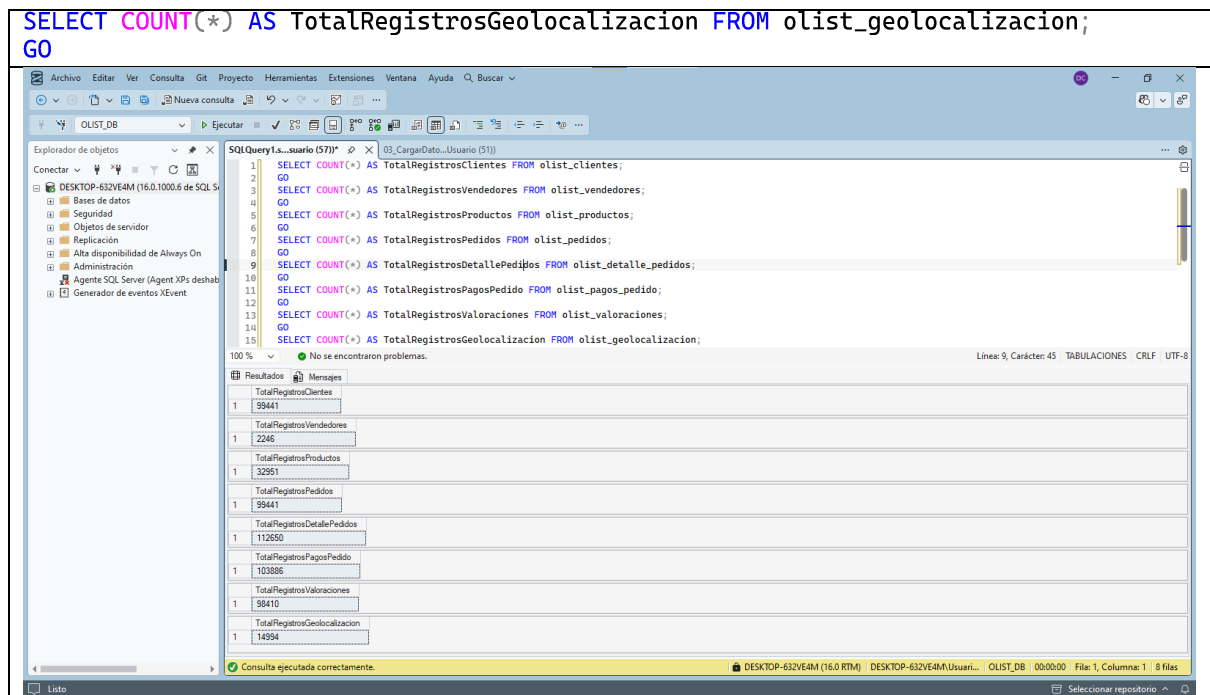
4.4. Número de registros cargados por tabla y los errores encontrados

Para simplificar se usa:

```
SELECT COUNT(<columna_o_asterisco>) AS <nombre_del_alias> FROM <nombre_de_tabla>;
GO
```

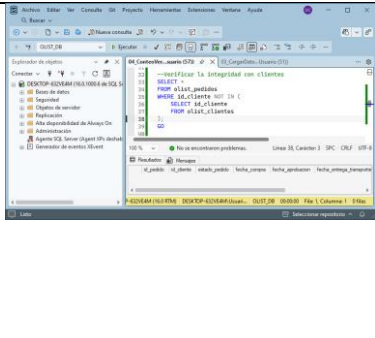
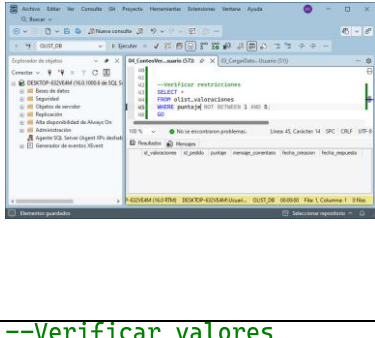
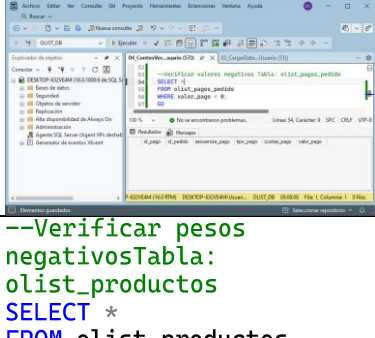
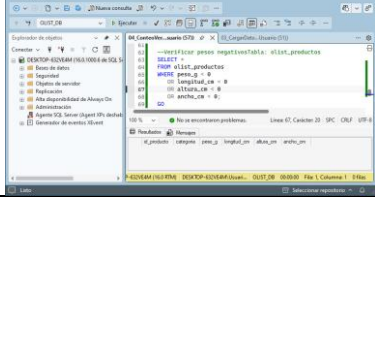
#	Tabla relacional	Registros Cargados en BD
1	olist_clientes	99441
2	olist_vendedores	2246
3	olist_productos	32951
4	olist_pedidos	99441
5	olist_detalle_pedidos	112650
6	olist_pagos_pedidos	103886
7	olist_valoraciones	98410
8	olist_geolocalizacion	14994

```
SELECT COUNT(*) AS TotalRegistrosClientes FROM olist_clientes;
GO
SELECT COUNT(*) AS TotalRegistrosVendedores FROM olist_vendedores;
GO
SELECT COUNT(*) AS TotalRegistrosProductos FROM olist_productos;
GO
SELECT COUNT(*) AS TotalRegistrosPedidos FROM olist_pedidos;
GO
SELECT COUNT(*) AS TotalRegistrosDetallePedidos FROM olist_detalle_pedidos;
GO
SELECT COUNT(*) AS TotalRegistrosPagosPedido FROM olist_pagos_pedido;
GO
SELECT COUNT(*) AS TotalRegistrosValoraciones FROM olist_valoraciones;
GO
```



Con el propósito de garantizar que la carga de datos se realizó correctamente, se ejecutaron diversas consultas SQL para verificar la integridad de la información almacenada en la base de datos. Las verificaciones incluyeron la búsqueda de registros duplicados en las llaves primarias, la validación de la integridad referencial entre tablas relacionadas y la comprobación del cumplimiento de las restricciones de integridad definidas durante el diseño del modelo relacional.

Verificación 1. Registros duplicados (Tabla olist_clientes)	<pre> --Verificacion de duplicados SELECT id_cliente, COUNT(*) AS Cantidad FROM olist_clientes GROUP BY id_cliente HAVING COUNT(*) > 1; GO </pre>	La consulta no devolvió registros, lo que indica que no existen identificadores de clientes duplicados en la tabla olist_clientes. En consecuencia, se confirma que la llave primaria mantiene la unicidad de los registros.
Verificación 2. Integridad referencial (Tabla olist_pedidos)	<pre> --Verificar la integridad con clientes SELECT * FROM olist_pedidos WHERE id_cliente NOT IN (SELECT id_cliente FROM olist_clientes); GO </pre>	No se encontraron pedidos asociados a clientes inexistentes. Esto confirma que la relación entre las tablas

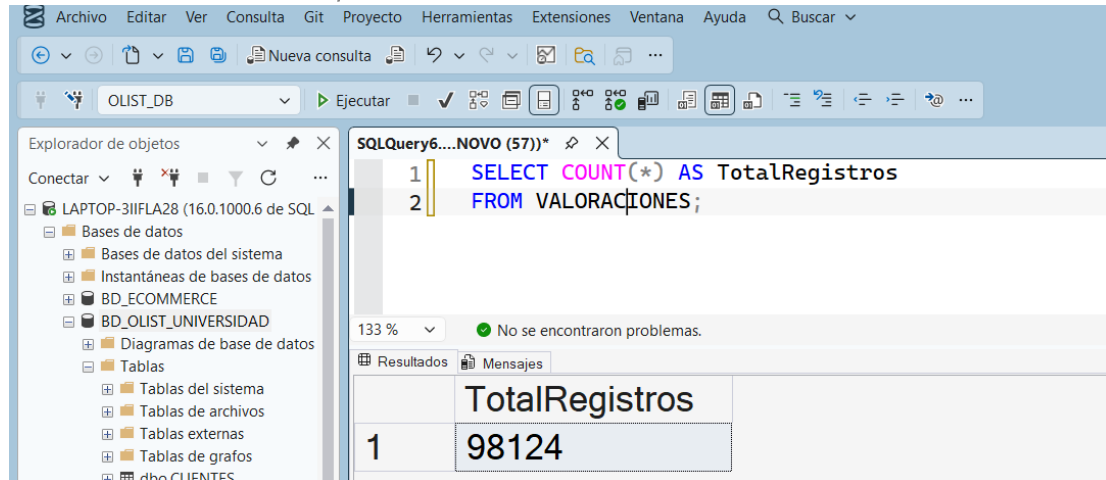
		olist_pedidos y olist_clientes se mantiene íntegra y que todas las claves foráneas hacen referencia a registros válidos.
Verificación 3. Restricción del puntaje (Tabla olist_valoraciones)	<pre>SELECT * FROM olist_valoraciones WHERE puntaje NOT BETWEEN 1 AND 5; GO</pre> 	La consulta no identificó registros con valores fuera del rango permitido (1 a 5), evidenciando que la restricción CHECK definida para la columna puntaje se cumple correctamente.
Verificación 4. Valores negativos (Tabla olist_pagos_pedido)	<pre>--Verificar valores negativos Tabla: olist_pagos_pedido SELECT * FROM olist_pagos_pedido WHERE valor_pago < 0; GO</pre> 	No se detectaron valores de pago negativos, lo que confirma la consistencia de los datos financieros almacenados en la tabla.
Verificación 5. Dimensiones de productos (Tabla olist_productos)	<pre>--Verificar pesos negativos Tabla: olist_productos SELECT * FROM olist_productos WHERE peso_g < 0 OR longitud_cm < 0 OR altura_cm < 0 OR ancho_cm < 0; GO</pre> 	No se encontraron registros con pesos o dimensiones negativas. Esto evidencia que los datos de los productos cumplen con las restricciones de integridad establecidas durante el modelado.

4.5. Reporte de calidad de datos

4.5.1. Calcular % de completitud

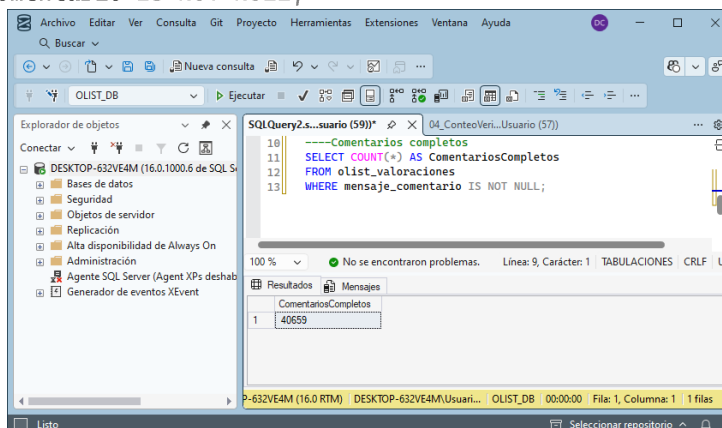
Total de registro

```
SELECT COUNT(*) AS TotalRegistros  
FROM olist_valoraciones;
```



Comentarios completos

```
SELECT COUNT(*) AS ComentariosCompletos  
FROM olist_valoraciones  
WHERE mensaje_comentario IS NOT NULL;
```



Fórmula

$$\begin{aligned} &= (\text{ComentariosCompletos} / \text{TotalRegistros}) \times 100 \\ &= (40659 / 98410) \times 100 \\ &= 41.31\% \end{aligned}$$

Interpretación

La completitud mide el porcentaje de registros que contienen información disponible en un determinado atributo. En este caso, se evaluó el campo mensaje_comentario de la tabla olist_valoraciones.

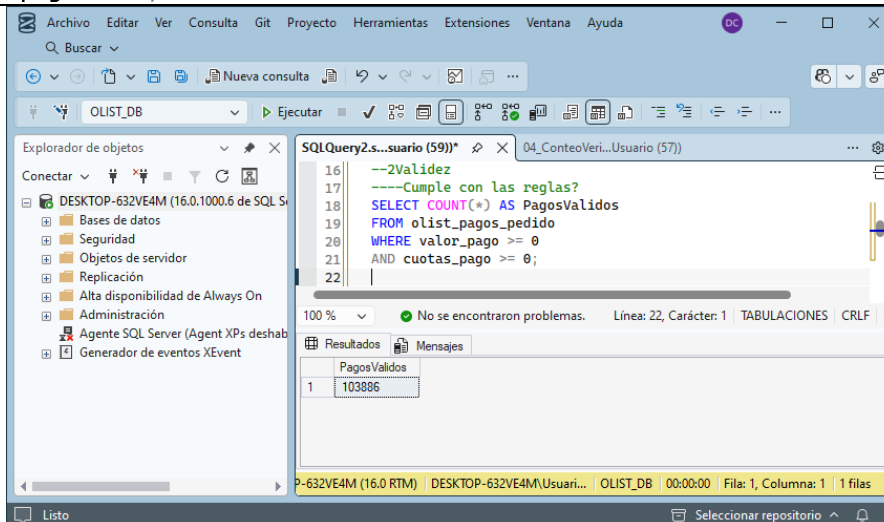
El resultado obtenido fue de 41.31%, lo que indica que 40,659 registros contienen comentarios escritos por los clientes, mientras que el porcentaje restante corresponde a registros donde los usuarios únicamente asignaron una calificación sin dejar una opinión textual. Debido a que el comentario es un campo opcional dentro del proceso de valoración,

este resultado es consistente con el comportamiento esperado del negocio y no representa un error en los datos.

4.5.2. Calcular % de pagos válidos

¿Los registros cumplen las reglas?

```
SELECT COUNT(*) AS PagosValidos
FROM olist_pagos_pedido
WHERE valor_pago >= 0
AND cuotas_pago >= 0;
```



Fórmula

$$= (\text{PagosValidos} / \text{TotalRegistros}) \times 100$$
$$= (103886 / 103886) \times 100$$
$$= 100\%$$

Interpretación

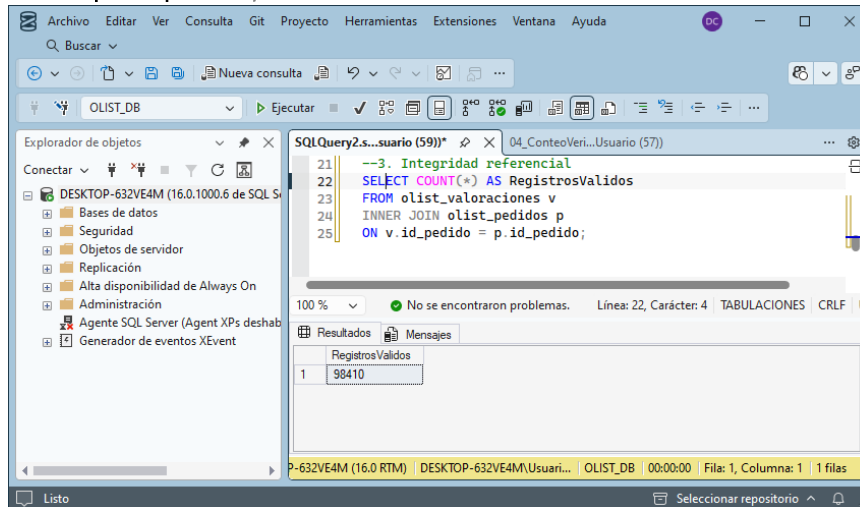
La validez mide el porcentaje de pagos que cumplen las reglas definidas para los datos. En este caso, se verificó que los valores del campo valor de pago sean mayor a 0 al igual que las cuotas de pago.

El resultado obtenido fue de 100%, lo que indica que todos los registros almacenados cumplen con las condiciones de validación establecidas. Esto demuestra que no existen valores negativos y que la información puede utilizarse de manera confiable para análisis posteriores.

4.5.3. Integridad referencial Valoraciones

Evalúa la asociación dentro de la base de datos

```
SELECT COUNT(*) AS RegistrosValidos
FROM olist_valoraciones v
INNER JOIN olist_pedidos p
ON v.id_pedido = p.id_pedido;
```



Total registros = 98410
Registros relacionados = 98410

100%

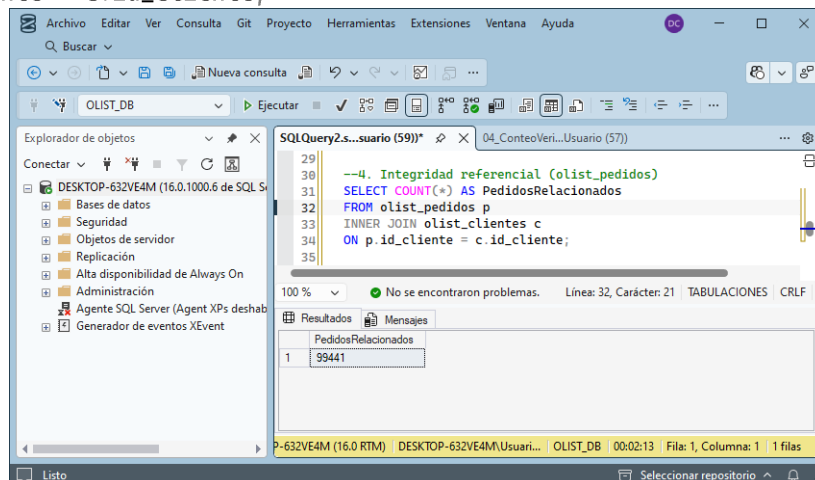
Interpretación

El 100% de las valoraciones posee un pedido asociado dentro de la base de datos, evidenciando que no existen registros huérfanos y que la integridad referencial entre ambas tablas se mantiene correctamente.

4.5.4. Integridad referencial Pedidos

Evalúa la asociación dentro de la base de datos

```
SELECT COUNT(*) AS PedidosRelacionados
FROM olist_pedidos p
INNER JOIN olist_clientes c
ON p.id_cliente = c.id_cliente;
```



Total registros = 99441
Registros relacionados = 99441

100%

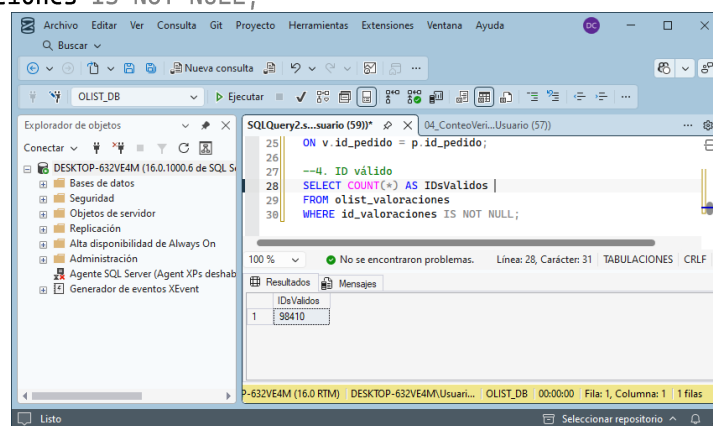
Interpretación

El 100% de los pedidos mantienen una relación válida con un cliente registrado, evidenciando que la integridad referencial de la base de datos se conserva correctamente.

4.5.5. Calcular % de registros con ID válido

Evalúa si cada registro tiene identificador

```
SELECT COUNT(*) AS IDValidos  
FROM olist_valoraciones  
WHERE id_valoraciones IS NOT NULL;
```



Fórmula

$$= (\text{Registros con ID válido} / \text{Total de registros}) * 100$$
$$= (98410 / 98410) * 100$$
$$= 100\%$$

Interpretación

Se verificó la integridad del identificador principal de la tabla VALORACIONES. El análisis mostró que el 100% de los registros posee un identificador válido y no nulo, garantizando la correcta identificación de cada valoración almacenada en la base de datos y evitando problemas de duplicidad o pérdida de trazabilidad de la información.

5. ETAPA 4 - Consultas SQL y Análisis Avanzado

5.1. Desarrollo de consultas para la toma de decisiones

¿Cuáles son las categorías de productos con mayor volumen de ventas?

```

151      select top 10 p.categoria,
152                  count(dp.id_producto) as [Volumen Ventas]
153      from olist_detalle_pedidos dp
154      inner join olist_productos p
155      on dp.id_producto = p.id_producto
156      group by p.categoria
157      order by [Volumen Ventas] desc;
158
159

```

78 % 9 0 ↑ ↓

Resultados Mensajes

	categoria	Volumen Ventas
1	cama_mesa_banho	11115
2	beleza_saude	9670
3	esporte_lazer	8641
4	moveis_decoracao	8334
5	informatica_acessorios	7827
6	utilidades_domesticas	6964
7	relogios_presentes	5991
8	telefonias	4545
9	ferramentas_jardim	4347
10	automotivo	4235

El resultado te mostrará el "Top 10" de lo que más mueve el e-commerce. Esto no habla de dinero, sino de volumen operativo. Demuestra qué categorías requieren mayor atención en almacenamiento, rotación de inventario y acuerdos con proveedores.

¿Qué estados o ciudades generan la mayor cantidad de pedidos?

```

161      select top 10 g.estado, g.ciudad,
162                  count(p.id_pedido) as [Total Pedidos]
163      from olist_pedidos p
164      inner join olist_clientes c
165      on p.id_cliente = c.id_cliente
166      inner join olist_geolocalizacion g
167      on c.codigo_postal = g.codigo_postal
168      group by g.estado, g.ciudad
169      order by [Total Pedidos] desc;
170
171

```

78 % 8 0 ↑ ↓

Resultados Mensajes

	estado	ciudad	Total Pedidos
1	SP	sao paulo	15540
2	RJ	rio de janeiro	6882
3	MG	belo horizonte	2773
4	DF	brasilia	2136
5	PR	curitiba	1521
6	SP	campinas	1444
7	RS	porto alegre	1379
8	BA	salvador	1245
9	SP	guarulhos	1189
10	SP	sao bernardo do campo	938

Esta información es vital para la toma de decisiones estratégicas, como dónde abrir un nuevo centro de distribución físico o hacia qué ciudades dirigir las campañas de marketing digital para maximizar el retorno de inversión.

¿Existe relación entre el tiempo de entrega y la satisfacción del cliente?

```

172      select v.puntaje AS Estrellas,
173              avg(datediff(day, p.fecha_compra, p.fecha_entrega_cliente)) as [Promedio Dias Entrega],
174              count(p.id_pedido) as [Cantidad Pedidos Evaluados]
175      from olist_pedidos p
176      inner join olist_valoraciones v
177      on p.id_pedido = v.id_pedido
178      where p.fecha_entrega_cliente > '2015-01-01'
179      group by v.puntaje
180      order by Estrellas desc;
181
182

```

78 % 8 0 ↑ ↓

Resultados Mensajes

	Estrellas	Promedio Dias Entrega	Cantidad Pedidos Evaluados
1	5	10	56658
2	4	12	18861
3	3	14	7885
4	2	16	2908
5	1	21	9295

Al cruzar la tabla de pedidos con las valoraciones, descubrimos una correlación inversa perfecta entre el tiempo de entrega y la satisfacción del cliente. Esta métrica nos demuestra que el principal cuello de botella para la fidelización de clientes en Olist no es la calidad del producto en sí, sino la eficiencia del transporte.

¿Qué vendedores registran mayores ingresos por ventas?

```

182
183
184
185
186
187
188
select top 10 id_vendedor,
              sum(precio) as [Ingresos Totales],
              count(id_producto) as [Cantidad Articulos Vendidos]
from olist_detalle_pedidos
group by id_vendedor
order by [Ingresos Totales] desc;

```

78 %

8 0 ↑ ↓

Resultados

Mensajes

	id_vendedor	Ingresos Totales	Cantidad Articulos Vendidos
1	4869f7a5dfa277a7dca6462dcf3b52b2	229472.63	1156
2	53243585a1d6dc2643021fd1853d8905	222776.05	410
3	4a3ca9315b744ce9f8e9374361493884	200472.92	1987
4	fa1c13f2614d7b5c4749cbc52fecda94	194042.03	586
5	7c67e1448b00f6e969d365cea6b010ab	187923.89	1364
6	7e93a43ef30c4f03f38b393420bc753a	176431.87	340
7	da8622b14eb17ae2831f4ac5b9dab84a	160236.57	1551
8	7a67c85e85bb2ce8582c35f2203ad736	141745.53	1171
9	1025f0e2d44d7041d6cf58b6550e0bfa	138968.55	1428
10	955fee9216a65b617aa5c0531780ce60	135171.70	1499

Identificación de socios comerciales estratégicos. Son vendedores top son el núcleo financiero del Marketplace, sin embargo, depender financieramente de un grupo tan reducido representa un riesgo operativo. Se recomienda analizar si estos vendedores tienen exclusividad con Olist; de no ser así, una fuga de estos socios a otra plataforma impactaría drásticamente los ingresos.

¿Qué porcentaje de pedidos fue entregado antes de la fecha estimada?

```

190
191
192
193
194
195
196
197
select
count(id_pedido) as [Total Pedidos Entregados],
sum(case when fecha_entrega_cliente <= fecha_estimada_entrega then 1 else 0 end) as [Entregados A Tiempo],
cast(sum(case when fecha_entrega_cliente <= fecha_estimada_entrega then 1 else 0 end) * 100.0 / count(id_pedido) as decimal(5,2)) as [Porcentaje Eficiencia]
from olist_pedidos
where fecha_entrega_cliente is not null
and fecha_estimada_entrega is not null;

```

78 %

8 0 ↑ ↓

Línea: 199, C

Resultados

Mensajes

	Total Pedidos Entregados	Entregados A Tiempo	Porcentaje Eficiencia
1	99441	91614	92.13

Al evaluar el desempeño de las entregas, la base de datos nos indica que, de un universo de 99,441 pedidos completados, 91,614 llegaron a las manos del cliente en la fecha prometida o incluso antes. Esto nos arroja un Porcentaje de Eficiencia Logística del 92.13%.

5.2. Desarrollo de consultas

Consulta 1: Identificación de la Élite de Vendedores

```
200 select
201     id_vendedor,
202     count(id_producto) as [Volumen Articulos],
203     sum(precio) as [Ingresos Totales]
204 from olist_detalle_pedidos
205 group by id_vendedor
206 having sum(precio) > 50000
207 order by [Ingresos Totales] desc;
```

78 % 8 0

id_vendedor	Volumen Articulos	Ingresos Totales
4869f7a5dfa277a7dca6462dcf3b52b2	1156	229472.63
53243585a1d6dc2643021fd1853d8905	410	222776.05
4a3ca9315b744ce9f8e9374361493884	1987	200472.92
fa1c13f2614d7b5c4749cbc52fecda94	586	194042.03
7c67e1448b00f6e969d365cea6b010ab	1364	187923.89
7e93a43ef30c4f03f38b393420bc753a	340	176431.87
da8622b14eb17ae2831f4ac5b9dab84a	1551	160236.57
7a67c85e85bb2ce8582c35f2203ad736	1171	141745.53
1025f0e2d44d7041d6cf58b6550e0bfa	1428	138968.55
955fee9216a65b617aa5c0531780ce60	1499	135171.70

Esta métrica aísla a los socios comerciales más rentables del ecosistema (aquellos que han facturado más de \$50,000). A nivel estratégico, estos vendedores deben recibir beneficios de fidelización (menores comisiones o soporte prioritario) porque sostienen financieramente la plataforma.

Consulta 2: Zonas de Mayor Demanda Logística (Múltiples INNER JOIN)

```
209 select top 10 g.estado, g.ciudad,
210     count(p.id_pedido) as [Total_Pedidos]
211 from olist_pedidos p
212 inner join olist_clientes c
213 on p.id_cliente = c.id_cliente
214 inner join olist_geolocalizacion g
215 on c.codigo_postal = g.codigo_postal
216 group by g.estado, g.ciudad
217 order by Total_Pedidos desc;
```

78 % 8 0

estado	ciudad	Total_Pedidos
SP	sao paulo	15540
RJ	rio de janeiro	6882
MG	belo horizonte	2773
DF	brasilia	2136
PR	curitiba	1521
SP	campinas	1444
RS	porto alegre	1379
BA	salvador	1245
SP	guarulhos	1189
SP	sao bernardo do campo	938

Mapea los centros urbanos con mayor adopción del e-commerce. Esto justifica ante la gerencia dónde se deben construir nuevos centros de distribución (Dark Stores) para reducir los costos de flete.

Consulta 3: Auditoría de Riesgo Logístico (Uso de LEFT JOIN)

```
220 select g.codigo_postal, g.ciudad, g.estado,
221     count(c.id_cliente) as Clientes_En_Riesgo_Logistico
222 from olist_geolocalizacion g
223 left join olist_clientes c
224 on g.codigo_postal = c.codigo_postal
225 where g.ciudad = 'Desconocido' or g.estado = 'NA'
226 group by g.codigo_postal, g.ciudad, g.estado
227 order by Clientes_En_Riesgo_Logistico desc;
```

78 % 12 0

codigo_postal	ciudad	estado	Clientes_En_Riesgo_Logistico
1001	Desconocido	NA	0
1039	Desconocido	NA	0
1040	Desconocido	NA	0
1126	Desconocido	NA	0
1212	Desconocido	NA	0
12331	Desconocido	NA	0
12913	Desconocido	NA	0
13154	Desconocido	NA	0
13328	Desconocido	NA	0
14078	Desconocido	NA	0

"La consulta identifica clientes ubicados en 'zonas ciegas', utilizando los valores estandarizados ('Desconocido' / 'NA') que aplicamos vía ETL para proteger la integridad relacional. A nivel de negocio, esto cuantifica un riesgo logístico crítico por alta probabilidad de devolución de paquetes. Como acción gerencial, se recomienda bloquear nuevas compras de estos usuarios hasta que completen una campaña obligatoria de actualización de dirección."

Consulta 4: Categorías Más Rentables del E-commerce (RIGHT JOIN)

```

230
231
232
233
234
235
236
237
238
239
240
select top 10 p.categoria,
count(dp.id_pedido) as [Unidades Vendidas],
sum(dp.precio) as [Volumen Ventas Monetario]
from olist_detalle_pedidos dp
right join olist_productos p
on dp.id_producto = p.id_producto
where p.categoria is not null
group by p.categoria
order by [Volumen Ventas Monetario] desc;

```

	categoria	Unidades Vendidas	Volumen Ventas Monetario
1	beleza_saude	9670	1258681.34
2	relogios_presentes	5991	1205005.68
3	cama_mesa_banho	11115	1036988.68
4	esporte_lazer	8641	988048.97
5	informatica_acessorios	7827	911954.32
6	moveis_decoracao	8334	729762.49
7	cool_stuff	3796	635290.85
8	utilidades_domesticas	6964	632248.66
9	automotivo	4235	592720.11
10	ferramentas_jardim	4347	485256.46

Esta consulta toma la totalidad de nuestros productos registrados y los cruza con el historial de transacciones para mostrarnos el Top 10 de las categorías que sostienen financieramente al e-commerce. A nivel estratégico, conocer nuestro 'Core Business' es vital.

Consulta 5: Comportamiento Transaccional y Recaudación (Agrupación Mixta)

```

235
236
237
238
239
240
241
select tipo_pago,
count(id_pedido) as [Frecuencia Uso],
sum(valor_pago) as [Volumen Dinero Movido],
cast((sum(valor_pago) * 100.0) / (select sum(valor_pago) from olist_pagos_pedido) as decimal(5,2)) as [Porcentaje Del Total]
from olist_pagos_pedido
group by tipo_pago
order by [Volumen Dinero Movido] desc;

```

	tipo_pago	Frecuencia Uso	Volumen Dinero Movido	Porcentaje Del Total
1	credit_card	76795	12542084.19	78.34
2	boleto	19784	2869361.27	17.92
3	voucher	5775	379436.87	2.37
4	debit_card	1529	217989.79	1.36
5	not_defined	3	0.00	0.00

Revela la dependencia del negocio frente a distintos medios de pago. Si la tarjeta de crédito mueve alrededor de 80% del dinero, Olist tiene un fuerte poder de negociación para exigir menores tasas de interés a los bancos emisores.

Consulta 6: Transacciones "Premium" (Subconsulta No Correlacionada)

```

244
245
246
247
248
249
select id_pedido, id_producto, precio
from olist_detalle_pedidos
where precio > (
select AVG(precio) from olist_detalle_pedidos
)
order by precio desc;

```

	id_pedido	id_producto	precio
1	0812eb902a67711a1cb742b3cdaa65ae	489ae2aa008f021502940f251d4cce7f	6735.00
2	fefacc66af859508bf1a7934eab1e97f	69c590f7ffc7bf8db97190b6cb6ed62e	6729.00
3	f5136e38d1a14a4dbd87dff67da82701	1bdf5e6731585cf01aa8169c7028d6ad	6499.00
4	a96610ab360d42a2e5335a3998b4718a	a6492cc69376c469ab6f61d8f44de961	4799.00
5	199af31afc78c699f0dbf71fb178d4d4	c3ed642d592594bb648f4a04cee2747	4690.00
6	8dbc85d1447242f3b127dda390d56e19	259037a6a41845e455183f89c5035f18	4590.00

Aísala las ventas que superan el "Ticket Promedio" de la plataforma. Analizar qué productos elevan el ticket promedio es vital para maximizar la rentabilidad con menos esfuerzo logístico.

Consulta 7: Vendedores que Rompen el Mercado (Subconsulta Correlacionada)

```

251      select dp.id_vendedor, dp.id_producto, dp.precio
252      from olist_detalle_pedidos dp
253      where dp.precio > (
254          select avg(dp2.precio)
255          from olist_detalle_pedidos dp2
256          where dp2.id_vendedor = dp.id_vendedor
257      )
258      order by dp.id_vendedor;
259

```

78 % 8 0 ↑ ↓

Resultados Mensajes

	id_vendedor	id_producto	precio
1	001cca7ae9ae17fb1caed9dfb1094831	e251ebd2858be1aa7d9b2087a6992580	112.00
2	001cca7ae9ae17fb1caed9dfb1094831	98a8c2fa16d7239c606640f555768e4	109.00
3	001cca7ae9ae17fb1caed9dfb1094831	547b95702aec86f05ac37e61d164891c	129.00
4	001cca7ae9ae17fb1caed9dfb1094831	e251ebd2858be1aa7d9b2087a6992580	112.00
5	001cca7ae9ae17fb1caed9dfb1094831	e251ebd2858be1aa7d9b2087a6992580	139.90
6	001cca7ae9ae17fb1caed9dfb1094831	e251ebd2858be1aa7d9b2087a6992580	112.00
7	001cca7ae9ae17fb1caed9dfb1094831	86b22a03cb72239dd53996a67df35c63	120.00

Análisis de dispersión de precios de un mismo proveedor. Detecta qué artículos le generan mayor margen individual a cada vendedor en comparación con su catálogo habitual, indicando sus productos estrella.

Consulta 8: Vista de Cumplimiento Logístico (SLA)

```

261      create view vw_kpi_eficiencia_logistica as
262      select
263          count(id_pedido) as [Total Pedidos Entregados],
264          sum(case when fecha_entrega_cliente <= fecha_estimada_entrega then 1 else 0 end) as [Entregados_A_Tiempo],
265          cast(sum(case when fecha_entrega_cliente <= fecha_estimada_entrega then 1 else 0 end) * 100.0 / count(id_pedido) as decimal(5,2)) as [Porcentaje Eficiencia]
266      from olist_pedidos
267      where fecha_entrega_cliente is not null
268          and fecha_estimada_entrega is not null;
269
270      select * from vw_kpi_eficiencia_logistica
271

```

78 % 9 0 ↑ ↓ Línea: 260, Cc

Resultados Mensajes

	Total Pedidos Entregados	Entregados_A_Tiempo	Porcentaje Eficiencia
1	99441	91614	92.13

Encapsulamiento del algoritmo del KPI logístico. Se empaqueta en una vista para que gerencia o herramientas como Power BI puedan consultar el estado de la eficiencia diaria sin tener que reescribir ni conocer la lógica transaccional compleja que hay detrás.

Consulta 9: Vista de Correlación Satisfacción-Demora

```

274      create view vw_impacto_demora_satisfaccion as
275      select v.puntaje as Estrellas,
276          avg(datediff(day, p.fecha_compra, p.fecha_entrega_cliente)) as [Promedio Dias Entrega],
277          count(p.id_pedido) as [Cantidad Pedidos Evaluados]
278      from olist_pedidos p
279      inner join olist_valoraciones v
280      on p.id_pedido = v.id_pedido
281      where p.fecha_entrega_cliente > '2015-01-01'
282      group by v.puntaje;
283
284      select * from vw_impacto_demora_satisfaccion
285

```

78 % 10 0 ↑ ↓

Resultados Mensajes

	Estrellas	Promedio Dias Entrega	Cantidad Pedidos Evaluados
1	1	21	9295
2	2	16	2908
3	3	14	7885
4	4	12	18861
5	5	10	56658

Mantiene disponible en tiempo real el modelo que relaciona los tiempos de entrega con la percepción del cliente, aislando las anomalías de datos históricos para una lectura siempre precisa.

Consulta 10: Buscador 360 de Salud Operativa del Vendedor

```

288 create procedure sp_Auditoria_Vendedor
289 @Id_Vendedor varchar(50)
290 as
291 begin
292     set nocount on;
293
294     select
295         v.id_vendedor,
296         count(dp.id_pedido) as [Total Ventas],
297         sum(dp.precio) as [Volumen Dinero],
298         avg(cast(val.puntaje as decimal(5,2))) as [Promedio Estrellas Recibidas]
299     from olist_vendedores v
300     inner join olist_detalle_pedidos dp
301     on v.id_vendedor = dp.id_vendedor
302     inner join olist_pedidos p
303     on dp.id_pedido = p.id_pedido
304     inner join olist_valoraciones val
305     on p.id_pedido = val.id_pedido
306     where v.id_vendedor = @Id_Vendedor
307     group by v.id_vendedor;
308 end;
309
310 select top 1 id_vendedor
311 from olist_detalle_pedidos;
312
313 exec sp_Auditoria_Vendedor @Id_Vendedor = '4869f7a5dfa277a7dca6462dcf3b52b2';

```

78 % 12 0

Resultados Mensajes

	id_vendedor	Total Ventas	Volumen Dinero	Promedio Estrellas Recibidas
1	4869f7a5dfa277a7dca6462dcf3b52b2	1146	227808.14	4.121291

Este es el producto final de la base de datos entregado al usuario final. Permite al área de operaciones ingresar el ID de cualquier vendedor y auditar de inmediato sus ventas, facturación y la calidad de sus productos (estrellas). Es el puente directo entre la arquitectura de datos y la toma de decisiones ágil.

Consulta 11: SELECT con filtros múltiples y ORDER BY

```

344 select id_pedido, id_cliente, estado_pedido, fecha_compra
345 from olist_pedidos
346 where (estado_pedido like 'cancel%' or estado_pedido = 'unavailable')
347 and fecha_compra between '2017-11-01' and '2017-12-31'
348 order by fecha_compra desc;
349 GO
350
351

```

84 % 13 0

Resultados Mensajes

	id_pedido	id_cliente	estado_pedido	fecha_compra
1	6ec704c9a120baaa6b3e16225ec684b	ea493c4332b771f1c1cf1592c320f8c9	unavailable	2017-12-30 10:52:00.000
2	50bb0c134ee3365dd06e13da6cda15fe	95cf2f1f5ef26bb509b276c5f7037646	unavailable	2017-12-29 16:31:00.000
3	7ecb6c38b166531663fdecce49dc1a6f	288f90dc3b55afaff2904553fa5cbfe0	unavailable	2017-12-29 00:18:00.000
4	8a20e2954602a0738f264299255fd5af	36ecc6ff2c4a59fd150e9524b12b9259	unavailable	2017-12-27 22:41:00.000
5	2efab00b70c96a73f5fed910d373957a	0b82ef8137cfb5f47ebcc41131c03663	canceled	2017-12-24 10:07:00.000
6	00d0ffd14774da775ac832ba8520510f	a2f3de2a0a84803cef018776a2e76c9b	canceled	2017-12-23 22:25:00.000
7	1c7c8ab324927f5b6e52338bd110bef4	95b1767df51d6fa09402267d8de314df	unavailable	2017-12-23 18:48:00.000
8	33e13c48e5883f2a3ae2269bf57650c7	cef54c3ad569af033c4ecdac69c9e140	unavailable	2017-12-22 12:41:00.000
9	1def7af24cb7182497acfd953f6dd88b	b35c38c2cd6a72b58cb104154d9782...	canceled	2017-12-21 09:22:00.000
10	2f5124f516a7f5e4095ef4325d67c511	2d2259e2fbd96d9aae631389a1d22cf6	unavailable	2017-12-19 20:36:00.000

A nivel comercial, los meses de noviembre y diciembre representan la temporada más crítica de ventas debido a campañas como *Black Friday* y *Navidad*. Esta consulta actúa como una auditoría de crisis operativa. Permite a la gerencia y al equipo de atención al cliente identificar de manera inmediata todos los pedidos que fracasaron (cancelados o sin stock disponible) durante esta temporada alta, con el objetivo de evaluar el impacto en los ingresos, medir las fallas de inventario y generar campañas de retención para los clientes afectados.

Consulta 12: Cortafuegos Operativo para Prevención de Pérdidas Logísticas (Uso de TRIGGER)

```

352 create trigger trg_Bloquear_Compra_Sin_Direccion
353 on olist_pedidos
354 after insert
355 as
356 begin
357     -- Verificamos si el cliente que intenta registrar un pedido está en una 'zona ciega'
358     if exists (
359         select 1
360         from inserted i
361         inner join olist_clientes c on i.id_cliente = c.id_cliente
362         inner join olist_geolocalizacion g on c.codigo_postal = g.codigo_postal
363         where g.ciudad = 'Desconocido' or g.estado = 'NA'
364     )
365     begin
366         -- Deshace la transacción inmediatamente y lanza un mensaje de error a la aplicación
367         rollback transaction;
368         raiserror ('Alerta: El cliente debe actualizar su dirección logística para proceder con la compra.', 16, 1);
369     end
370 end;
371 GO
372
373 /* PRUEBA DE CORTAFUEGOS LOGÍSTICO (TRIGGER) */
374 -- PASO 1: Simulamos el registro de un NUEVO CLIENTE.
375 INSERT INTO olist_clientes (id_cliente, id_unico_cliente, codigo_postal)
376 VALUES ('CLI-RIESGO-999', 'UNI-RIESGO-999', '1001');
377 GO
378
379 -- Comprobamos que el cliente está creado en la base de datos:
380 SELECT * FROM olist_clientes WHERE id_cliente = 'CLI-RIESGO-999';
381 GO
382
383 -- PASO 2: El cliente intenta comprar (AQUÍ ENTRA EN ACCIÓN EL TRIGGER).
384 INSERT INTO olist_pedidos (id_pedido, id_cliente, estado_pedido, fecha_compra)
385 VALUES ('PEDIDO-TEST-001', 'CLI-RIESGO-999', 'invoiced', '2018-05-01');
386 GO
387

```

63 %
13
0
↑ ↓

Mensajes

Message 30000, nivel 16, estado 1, procedimiento trg_Bloquear_Compra_Sin_Direccion, línea 17 (línea de inicio de lote 354)

Alerta: El cliente debe actualizar su dirección logística para proceder con la compra.

Msgs. 3000, Nivel 16, Estado 1, Línea 17

The transaction ended in the trigger. The batch has been aborted.

```

388 -- PASO 3: Limpieza del registro de prueba (Mantenimiento post-auditoria)
389 DELETE FROM olist_clientes
390 WHERE id_cliente = 'CLI-RIESGO-999';
391 GO
392

```

63 %
13
0
↑ ↓

Mensajes

(1 fila afectada)

hora de finalización: 2028-07-08T18:10:22.0541403-05:00

En el sector e-commerce, enviar un paquete a una dirección inválida genera altísimos sobrecostos por fletes perdidos y logística inversa (devoluciones). Este código funciona como un "cortafuegos operativo": automatiza una regla de negocio estricta directamente en la base de datos, rechazando transacciones riesgosas y obligando al usuario a actualizar sus datos. Con esto, la empresa previene pérdidas financieras automatizando el control de calidad en su distribución logística.

6. ETAPA 5 - Big Data e Integración de Resultados

6.1. Análisis de las 5 V del Big Data

Dimensión	Definición	Respondiendo a las preguntas	Conclusión
-----------	------------	------------------------------	------------

Volumen	El dataset OLIST está conformado por ocho tablas relacionadas que representan las operaciones de una plataforma de comercio electrónico: clientes, pedidos, detalle de pedidos, productos, vendedores, pagos, valoraciones y geolocalización. Según el proceso ETL documentado en el proyecto, se cargaron 563 817 registros en SQL Server, distribuidos de la siguiente manera: Clientes (99 441), Pedidos (99 441), Detalle de pedidos (112 650), Pagos (103 886), Valoraciones (98 410), Productos (32 951), Vendedores (2 246) y Geolocalización (14 994).	¿Cuántos registros tiene el dataset? El conjunto de datos utilizado en el proyecto contiene 563 817 registros distribuidos en ocho tablas. ¿A nivel nacional y en tiempo real cuántos registros se generarían por día? Si este modelo fuera implementado en una plataforma real de comercio electrónico, diariamente podrían generarse cientos de miles o millones de nuevos registros, considerando nuevas compras, pagos, envíos, actualizaciones logísticas y valoraciones de clientes.	El volumen es considerable para un proyecto académico y permite realizar análisis complejos mediante SQL Server. Sin embargo, todavía no alcanza el volumen masivo que caracteriza a un entorno Big Data empresarial.
Variedad	El dataset integra información correspondiente a diferentes procesos del negocio: clientes, pedidos, productos, pagos, vendedores, geolocalización y valoraciones. Además, almacena distintos tipos de datos como identificadores, números enteros, valores monetarios, fechas, códigos postales y comentarios escritos por los clientes. Durante el proceso ETL también fue necesario normalizar formatos de fecha y conservar caracteres especiales mediante codificación UTF-8.	¿El dataset contiene solo datos estructurados? No, aunque la mayor parte de la información está organizada en tablas relacionales, también incluye comentarios de clientes (texto libre), considerados datos semiestructurados. ¿Qué datos no estructurados podrían enriquecer el análisis? Fotografías de productos, imágenes de reclamos, audios del servicio de atención, videos, publicaciones en redes sociales, documentos electrónicos y registros de	El dataset presenta una alta variedad de datos, ya que combina información transaccional, geográfica y textual. La incorporación de datos multimedia permitiría realizar análisis más avanzados mediante técnicas de inteligencia artificial.

		navegación de los usuarios.	
Velocidad	La información corresponde a datos históricos recopilados entre 2016 y 2018 y descargados desde Kaggle. El procesamiento se realizó mediante un proceso ETL por lotes (batch), donde primero se extrajeron los archivos CSV, luego se transformaron y finalmente se cargaron en SQL Server.	¿Con qué frecuencia se actualiza este dataset? No se actualiza automáticamente, ya que corresponde a un conjunto de datos histórico. ¿Podría procesarse en tiempo real (streaming)? Sí, ya que, si la plataforma estuviera en funcionamiento, los pedidos, pagos y estados logísticos podrían procesarse en tiempo real utilizando tecnologías como Apache Kafka o Apache Spark Streaming.	Actualmente el dataset no cumple con la característica de velocidad del Big Data porque trabaja sobre información histórica y procesamiento por lotes; sin embargo, su estructura permitiría adaptarse a un entorno de procesamiento en tiempo real.
Veracidad	La calidad de los datos fue garantizada mediante el proceso ETL, que incluyó limpieza de registros, tratamiento de valores nulos, conversión de fechas, codificación UTF-8, validaciones mediante restricciones SQL y normalización de la base de datos. El reporte de calidad muestra 100 % de registros válidos, 100 % de identificadores válidos, 41,27 % de comentarios completos y 58,73 % de comentarios nulos, estos últimos considerados normales porque el comentario era opcional.	¿Qué porcentaje de registros presenta inconsistencias o valores nulos? No existen inconsistencias en identificadores ni puntajes. El único valor nulo significativo corresponde al 58,73 % de comentarios, situación esperada debido a que los clientes podían calificar sin escribir una opinión. ¿Cómo se mitigaron? Mediante procesos ETL de limpieza, validación de formatos, restricciones de integridad, normalización y conservación controlada de valores NULL cuando representaban el comportamiento normal del negocio.	La información posee un alto nivel de confiabilidad, permitiendo realizar análisis y consultas con resultados consistentes y útiles para la toma de decisiones.
Valor	El dataset permitió desarrollar consultas SQL para identificar las	¿Qué decisiones concretas puede tomar	El análisis demuestra que el dataset genera un alto valor

	categorías con mayor volumen de ventas, los vendedores con mayores ingresos, las ciudades con mayor demanda, la eficiencia logística, los métodos de pago más utilizados y la relación entre el tiempo de entrega y la satisfacción del cliente. Estas consultas proporcionan información estratégica para apoyar la gestión empresarial.	una organización con este análisis? Optimizar inventarios, mejorar la distribución logística, fortalecer relaciones con los vendedores más rentables, diseñar campañas de marketing más efectivas, abrir nuevos centros de distribución y mejorar la experiencia del cliente. ¿Qué beneficio económico o social genera? Permite reducir costos operativos, incrementar la rentabilidad, optimizar los tiempos de entrega, mejorar la satisfacción de los clientes y facilitar la toma de decisiones basada en datos.	estratégico, ya que convierte los datos operativos en información útil para mejorar el desempeño comercial, logístico y financiero de la empresa.
--	--	--	--

Después de analizar el dataset bajo las cinco dimensiones del Big Data, se concluye que el conjunto de datos OLIST no puede clasificarse como Big Data en su estado actual. Aunque posee una alta variedad, una elevada veracidad y un gran valor para la toma de decisiones, el volumen de 563 817 registros y el hecho de que se trate de información histórica permiten que sea administrado eficientemente mediante una base de datos relacional como SQL Server.

No obstante, sí podría formar parte de un entorno Big Data a escala real si la plataforma operara con millones de transacciones diarias, integrara datos no estructurados (imágenes, videos, redes sociales y registros de navegación) y utilizara tecnologías distribuidas como Apache Spark, Hadoop HDFS o Google BigQuery para procesar la información de forma continua y en tiempo real. De esta manera, cumpliría plenamente con las cinco dimensiones que caracterizan a un ecosistema Big Data.

6.2. Implementación de herramientas de Big Data

Para procesar el dataset OLIST a una escala mayor a la actual, se investigaron y describen a continuación dos herramientas del ecosistema Big Data que podrían aplicarse al proyecto.

6.2.1. Apache Spark

Apache Spark es un motor de procesamiento distribuido de código abierto diseñado para el análisis de grandes volúmenes de datos (Macías et al., 2015). Realiza sus operaciones principalmente en memoria RAM, lo que reduce los tiempos de ejecución en tareas de transformación y agregación frente a motores basados en disco. Es compatible con varios lenguajes de programación como Python (PySpark), Scala, Java y R, y ofrece el módulo Spark SQL, que permite ejecutar consultas con sintaxis similar a SQL sobre grandes conjuntos de datos distribuidos.

Spark funciona bajo una arquitectura maestro-trabajador (driver-executors), donde el nodo driver coordina la ejecución del programa y distribuye las tareas entre los nodos worker, que procesan en paralelo particiones del dataset. Trabaja sobre estructuras llamadas RDD (Resilient Distributed Datasets) y DataFrames, que permiten tolerancia a fallos y procesamiento distribuido de forma transparente para el desarrollador (Macías et al., 2015).

Frente a un RDBMS tradicional como el utilizado en este proyecto, Spark presenta la ventaja de escalar de forma horizontal, agregando más nodos al clúster conforme el volumen de datos crece, algo que un motor relacional no puede hacer con la misma facilidad porque su procesamiento está limitado a los recursos de un solo servidor. Para el caso de OLIST, esto permitiría distribuir el cálculo de agregaciones masivas, como el total de ingresos por vendedor o el análisis de tiempos de entrega a nivel nacional, entre varios nodos trabajando en paralelo, algo que en un RDBMS tradicional se ejecutaría de forma secuencial y con mayor tiempo de respuesta a medida que crece el volumen de datos. También facilita el procesamiento de datos no estructurados, como los comentarios de las valoraciones, mediante técnicas de procesamiento de texto o machine learning dentro del mismo ecosistema, algo que un RDBMS no está diseñado para hacer de forma nativa.

Como limitación, Spark requiere infraestructura adicional en forma de un clúster con varios nodos, lo que implica un costo de configuración y mantenimiento, y no está diseñado para transacciones individuales de lectura y escritura frecuentes como un sistema OLTP.

6.2.2. Google BigQuery

Google BigQuery es un data warehouse serverless alojado en Google Cloud, orientado a consultas analíticas sobre grandes volúmenes de datos (Lakshmanan & Tigani, 2020). El usuario carga los datos y ejecuta consultas en SQL estándar, mientras Google administra automáticamente los recursos de cómputo según la complejidad de cada consulta, sin necesidad de configurar o mantener servidores.

Su arquitectura separa el almacenamiento del cómputo, los datos se guardan en un almacenamiento columnar distribuido y cada consulta activa de forma dinámica la

cantidad de nodos de procesamiento necesarios para resolverla, liberándolos al finalizar. Esto permite escalar el análisis sin planificación previa de capacidad (Lakshmanan & Tigani, 2020).

Frente a un RDBMS (Sistema de Gestión de Bases de Datos Relacionales) tradicional, BigQuery tiene la ventaja de separar el almacenamiento del cómputo, mientras que en un motor relacional convencional ambos recursos están acoplados en el mismo servidor y deben escalarse juntos, aunque solo se necesite más capacidad de procesamiento. Para el caso de OLIST, BigQuery sería útil para ejecutar consultas analíticas recurrentes sobre históricos de ventas a nivel nacional, por ejemplo, reportes mensuales de ingresos por estado o categoría de producto, sin que ese procesamiento analítico compita por recursos con las operaciones transaccionales diarias del sistema, como sí ocurriría en un RDBMS tradicional. Además, se integra con herramientas de visualización como Google Looker Studio para generar dashboards directos sobre los resultados.

Como limitación, su modelo de costos se basa en el volumen de datos escaneados por cada consulta, por lo que un mal diseño de particiones o consultas que no filtren columnas específicas puede elevar el costo de uso. Al ser un servicio propietario de Google Cloud, además genera dependencia del proveedor.

6.3. Consulta en PySpark vs Pandas

Para validar de forma práctica el comportamiento de un motor tradicional en memoria frente a una arquitectura de Big Data, se diseñó un entorno de pruebas en Google Colab. Se implementaron dos escenarios analíticos operados bajo la librería Pandas (procesamiento secuencial en memoria RAM local) y PySpark (procesamiento distribuido basado en la arquitectura driver-executors).

El dataset utilizado para ambas pruebas fue el histórico transaccional centralizado en **olist_pedidos.csv**.

```
[1] # 1. Instalar PySpark en el entorno de Colab
✓ 6 s !pip install pyspark -q

import time
import pandas as pd
from pyspark.sql import SparkSession

print("Librerías instaladas y cargadas correctamente.")

... Librerías instaladas y cargadas correctamente.

[10] # Archivo CSV subido
nombre_archivo = "/olist_pedidos.csv"
print(f"Archivo a procesar: {nombre_archivo}")

... Archivo a procesar: /olist_pedidos.csv
```

6.3.1. Consulta 1: Conteo por estado de pedido

El objetivo de esta primera prueba fue medir la latencia base de ambos motores al ejecutar una operación aritmética simple de agrupación (GROUP BY + COUNT) sobre la columna Estado_pedido.

PANDAS	PYSPARK
<pre>[16] ✓ 0s print("--- INICIANDO PRUEBA CON PANDAS (Memoria RAM) ---") start_pandas = time.time() # Cargar y consultar agrupando por estado del pedido df_pandas = pd.read_csv(nombre_archivo) resultado_pandas = df_pandas.groupby('Estado_pedido').size() end_pandas = time.time() tiempo_pandas = end_pandas - start_pandas print(f"Tiempo total con Pandas: {tiempo_pandas:.4f} segundos") print("\n--- RESULTADO DE LA CONSULTA ---") print(resultado_pandas) --- INICIANDO PRUEBA CON PANDAS (Memoria RAM) --- Tiempo total con Pandas: 0.3702 segundos --- RESULTADO DE LA CONSULTA --- Estado_pedido approved 2 canceled 625 created 5 delivered 96478 invoiced 314 processing 301 shipped 1107 unavailable 609 dtype: int64</pre>	<pre>[16] ✓ 1s print("--- INICIANDO PRUEBA CON PYSPARK (Procesamiento Distribuido) ---") # Iniciar sesión Spark (El nodo Driver) spark = SparkSession.builder.appName("OlistTest").getOrCreate() start_spark = time.time() # Cargar, agrupar y ejecutar la acción df_spark = spark.read.csv(nombre_archivo, headers=True, inferSchema=True) resultado_spark = df_spark.groupby("Estado_pedido").count() resultado_spark.collect() # Esto detiene el procesamiento en los nodos end_spark = time.time() tiempo_spark = end_spark - start_spark print(f"Tiempo total con PySpark: {tiempo_spark:.4f} segundos") print("\n--- RESULTADO DE LA CONSULTA ---") resultado_spark.show() --- INICIANDO PRUEBA CON PYSPARK (Procesamiento Distribuido) --- Tiempo total con PySpark: 3.8767 segundos --- RESULTADO DE LA CONSULTA --- +-----+ Estado_pedido count +-----+ shipped 1107 canceled 625 approved 2 invoiced 314 created 5 delivered 96478 unavailable 609 processing 301 +-----+</pre>

6.3.2. Consulta 2: Top 5 fechas con mayor volumen

Para evaluar la capacidad de respuesta ante consultas más exigentes, se ejecutó una operación de filtrado, agrupación y ordenamiento descendente (SORT) sobre la columna Fecha_compra. Esta operación fuerza la reorganización física de los índices en memoria.

PANDAS	PYSPARK
<pre>print("--- PANDAS: TOP 5 DÍAS CON MÁS PEDIDOS ---") start_pandas_2 = time.time() # Agrupar por fecha, contar, ordenar de mayor a menor y sacar los 5 primeros top_fechas_pd = df_pandas.groupby('Fecha_compra').size().sort_values(ascending=False).head(5) end_pandas_2 = time.time() print(f"Tiempo Pandas: (end_pandas_2 - start_pandas_2:.4f) segundos") print(top_fechas_pd) --- PANDAS: TOP 5 DÍAS CON MÁS PEDIDOS --- Tiempo Pandas: 0.9985 segundos Fecha_compra 2/08/2018 12:06 28 2/08/2018 12:05 26 2/08/2018 12:07 22 20/11/2017 11:46 20 20/11/2017 12:47 13 dtype: int64</pre>	<pre>print("--- PYSPARK: TOP 5 DÍAS CON MÁS PEDIDOS ---") start_spark_2 = time.time() # Agrupar por fecha, contar, ordenar de forma descendente y limitar a 5 top_fechas_sp = df_spark.groupby("Fecha_compra").count().sort("count", ascending=False).limit(5) top_fechas_sp.collect() # Detonar el procesamiento end_spark_2 = time.time() print(f"Tiempo PySpark: (end_spark_2 - start_spark_2:.4f) segundos") top_fechas_sp.show() --- PYSPARK: TOP 5 DÍAS CON MÁS PEDIDOS --- Tiempo PySpark: 2.3906 segundos +-----+ Fecha_compra count +-----+ 2/08/2018 12:06 28 2/08/2018 12:05 26 2/08/2018 12:07 22 20/11/2017 11:46 20 20/11/2017 10:59 13 +-----+</pre>

6.3.3. Análisis comparativo

La elección entre Pandas y PySpark no radica en cuál motor es absolutamente mejor, sino en el volumen de información que el proyecto exige procesar. Pandas demuestra ser excepcionalmente rápido y eficiente para análisis exploratorios de tamaño moderado (Small Data), ya que opera directamente sobre la memoria RAM local con nula latencia de arranque; sin embargo, su capacidad está estrictamente limitada a la memoria de un solo equipo. Por el contrario, aunque PySpark requiere un tiempo

adicional de inicialización para configurar su entorno, se vuelve una herramienta indispensable en escenarios de Big Data. Su verdadera fortaleza tecnológica es la capacidad de escalar horizontalmente, distribuyendo cargas masivas de trabajo a través de múltiples servidores en paralelo para procesar millones de registros sin riesgo de colapsar la memoria, garantizando así la estabilidad de operaciones a nivel empresarial.

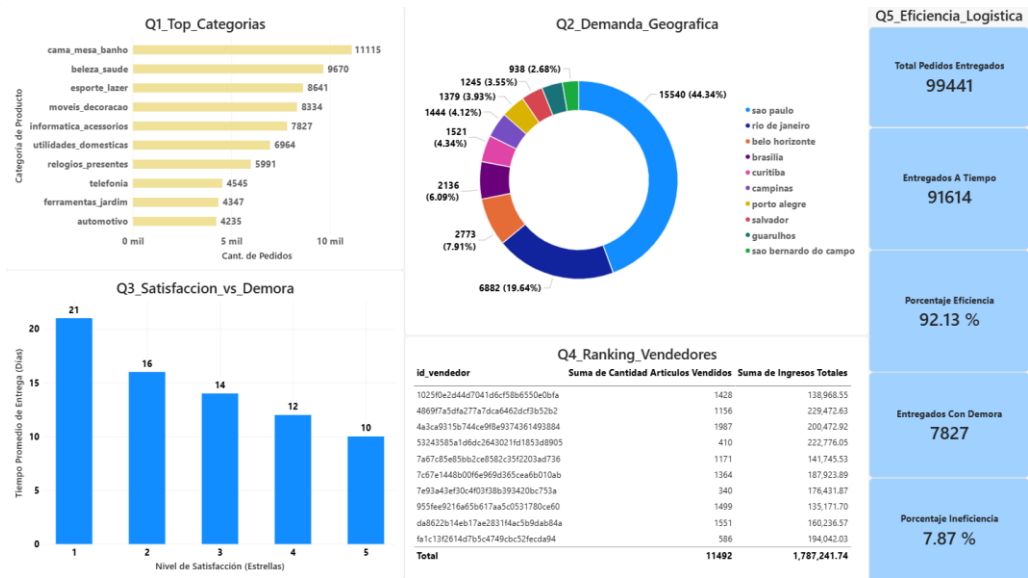
6.4. Dashboard en Power BI

Visión 360° del E-Commerce OLIST

El objetivo de este panel de control es proporcionar a la Alta Gerencia una herramienta visual e interactiva que responda a las cinco interrogantes estratégicas planteadas al inicio del proyecto. Este reporte centraliza indicadores de ventas, desempeño logístico y satisfacción del cliente, facilitando la toma de decisiones basada en datos empíricos.

Para la construcción de este reporte, se estableció una conexión directa entre Power BI Desktop y Microsoft SQL Server. El dashboard extrae la información consumiendo las consultas y métricas desarrolladas previamente en la base de datos, garantizando la integridad referencial y transaccional.

Evidencia Visual del Panel de Control



Interpretación de Indicadores y Toma de Decisiones

El análisis visual del dashboard nos arroja las siguientes conclusiones estratégicas para el negocio:

- Eficiencia Logística (Q5 - Tarjetas KPI):** El indicador más importante del panel demuestra un nivel de servicio (SLA) muy saludable. De un total histórico de 99,441

pedidos, el 92.13% (91,614 pedidos) fueron entregados a tiempo. Solo existe un 7.87% de ineficiencia operativa, lo que demuestra solidez en la red de distribución actual.

2. **Correlación de Satisfacción (Q3 - Columnas):** El gráfico confirma una correlación inversa perfecta entre los días de entrega y la calificación del cliente. Se evidencia que cuando los envíos promedian los 21 días, la calificación cae a 1 estrella, mientras que las entregas eficientes (10 días promedio) garantizan las 5 estrellas. Decisión estratégica: Establecer alertas en el sistema para cualquier envío que supere los 14 días (umbral crítico antes de caer a valoraciones negativas).
3. **Concentración de Demanda (Q2 - Anillo):** El área geográfica está fuertemente centralizada. Tan solo el estado de São Paulo (44.34%) y Río de Janeiro (19.64%) concentran más del 60% de la operación total. Decisión estratégica: Priorizar la construcción de nuevos centros de distribución físicos (Dark Stores) exclusivamente en estas dos zonas para seguir reduciendo los tiempos de entrega.
4. **Enfoque Comercial (Q1 y Q4):** A nivel de inventario, el Marketplace depende fuertemente de la categoría de "Cama, Mesa y Baño" (más de 11 mil pedidos). A nivel financiero, el top de vendedores recauda volúmenes superiores a los 1.7 millones en ingresos. Decisión estratégica: El área comercial debe ofrecer programas de fidelización y comisiones preferenciales a este grupo selecto de vendedores para evitar que migren a plataformas competidoras.

7. Conclusiones

1. El desarrollo del proyecto permitió aplicar de manera integrada los conocimientos adquiridos durante el curso, abarcando el modelado de bases de datos, la normalización hasta la Tercera Forma Normal (3FN), la implementación de una base de datos relacional, el proceso ETL, la elaboración de consultas SQL avanzadas y el análisis de calidad de datos. Esto fortaleció las competencias del equipo en el diseño e implementación de soluciones orientadas a la gestión de información.
2. La base de datos desarrollada permite organizar, almacenar y consultar de forma eficiente la información del conjunto de datos Olist E-commerce. Mediante las consultas SQL implementadas es posible obtener información relevante para apoyar la toma de decisiones relacionadas con clientes, pedidos, productos, vendedores y pagos, demostrando el valor que una adecuada gestión de datos aporta a una organización.
3. El proceso ETL permitió extraer, transformar y cargar correctamente los datos provenientes de los archivos CSV. Asimismo, las verificaciones realizadas evidenciaron que la información mantiene un adecuado nivel de integridad, consistencia y calidad, al no presentar registros duplicados en las llaves principales, valores fuera de las restricciones establecidas ni inconsistencias en las relaciones entre las tablas.
4. Como limitación, el proyecto se desarrolló utilizando un dataset estático, por lo que la información analizada no se actualiza en tiempo real. Además, algunas columnas

del conjunto de datos contenían valores opcionales o incompletos, lo que requirió realizar procesos de limpieza y validación durante la etapa de transformación para garantizar la calidad de la información.

5. Como trabajo futuro, se propone automatizar completamente el proceso ETL mediante herramientas especializadas como SQL Server Integration Services (SSIS) o Apache NiFi, así como implementar dashboards interactivos en plataformas como Power BI para facilitar la visualización de indicadores. Asimismo, el proyecto podría ampliarse utilizando tecnologías Big Data como Apache Spark o Google BigQuery para procesar volúmenes de información considerablemente mayores.

8. Anexo

8.1. Conformación de Equipos y Roles

Rol	Responsabilidades Principales	Entregable clave
Líder de Proyecto – Grace Vasquez Ipanaque	Coordina el equipo, gestiona tiempos, presenta en las exposiciones	Cronograma y actas de reunión
Arquitecto de BD - Chavez Durand, Sofia	Diseña el MER, el esquema relacional y la normalización	Diagrama MER y DDL de creación
Desarrollador SQL - Zarzosa Ramos, Hoberths	Escribe las consultas, vistas, funciones y procedimientos almacenados	Script SQL completo y documentado
Analistas de Datos - Jacay Ramos, Matias	Interpreta los resultados, elabora gráficas y conclusiones	Dashboard o informe de hallazgos
Especialistas ETL / Big Data - Quispe Córdova, Andrea	Realiza la limpieza, transformación y carga de datos; documenta el proceso ETL	Script ETL y reporte de calidad de datos

9. Referencias Bibliográficas

Kaggle. (s. f.). Brazilian E-Commerce Public Dataset by Olist. Kaggle.

<https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

Amazon Web Services. (s. f.). ¿Qué es ETL (extract, transform, load)? AWS.

<https://aws.amazon.com/es/what-is/etl/>

DataSunrise. (s. f.). DDL: Lenguaje de definición de datos. DataSunrise.

<https://www.datasunrise.com/es/centro-de-conocimiento/ddl-lenguaje-de-definicion-de-datos/>

EBAC. (s. f.). Consultas SQL. EBAC. <https://ebac.mx/blog/consultas-sql>

freeCodeCamp. (s. f.). Normalización de base de datos: Formas normales 1NF, 2NF y 3NF. freeCodeCamp.

<https://www.freecodecamp.org/espanol/news/normalizacion-de-base-de-datos-formas-normales-1nf-2nf-3nf-ejemplos-de-tablas/>

GeeksforGeeks. (s. f.). Relational model in DBMS. GeeksforGeeks.

<https://www.geeksforgeeks.org/dbms/relational-model-in-dbms/>

IBM. (s. f.). ¿Qué es una base de datos? IBM. [https://www.ibm.com/es-](https://www.ibm.com/es-es/think/topics/database)

[es/think/topics/database](https://www.ibm.com/es-es/think/topics/database)

Ilerna. (s. f.). Modelo entidad-relación en bases de datos. Ilerna.

<https://www.ilerna.es/blog/modelo-entidad-relacion-base-datos>

Intelequia. (s. f.). Qué es Microsoft SQL Server y para qué sirve. Intelequia.

<https://intelequia.com/es/blog/post/qu%C3%A9-es-microsoft-sql-server-y-para-qu%C3%A9-sirve>

OpenWebinars. (s. f.). Datasets: qué son y cómo acceder a ellos. OpenWebinars.

<https://openwebinars.net/blog/datasets-que-son-y-como-acceder-a-ellos/>

PingCAP. (s. f.). Primary key vs. foreign key: Unlocking efficient data management

in databases. PingCAP. <https://www.pingcap.com/article/primary-key-vs-foreign-key-unlocking-efficient-data-management-in-databases/>

The Knowledge Academy. (s. f.). Data cleaning: Definition, importance, steps and examples. The Knowledge Academy.

<https://www.theknowledgeacademy.com/blog/data-cleaning/>

UNIR. (s. f.). ¿Qué es SQL? Universidad Internacional de La Rioja.

<https://peru.unir.net/revista/ingenieria/que-es-sql/>

Webempresa. (s. f.). ¿Qué es SQL y para qué sirve? Webempresa.

<https://www.webempresa.com/blog/que-es-sql-y-para-que-sirve.html>

Lakshmanan, V., & Tigani, J. (2020). Google BigQuery: The definitive guide: Data warehousing, analytics, and machine learning at scale. O'Reilly Media.

https://books.google.com.pe/books/about/Google_BigQuery_The_Definitive_Guide.html?id=-Jq4DwAAQBAJ&redir_esc=y

Macías, M., Gómez, M., Tous, R., & Torres, J. (2015). Introducción a Apache Spark: Para empezar a programar el Big Data. Editorial UOC.

https://books.google.es/books?hl=es&lr=lang_es&id=JqunDAAAQBAJ&oi=fnd&pg=PT6&ots=fbwA6gO_e&sig=86_3FSk2SqNbR3jy7FZmDWerKI#v=onepage&q&f=false

Aguilar, L. J. (2016). Big Data, Análisis de grandes volúmenes de datos en organizaciones. Alfaomega Grupo

Editor.https://books.google.es/books?hl=es&lr=&id=1GywDAAAQBAJ&oi=fnd&pg=PT17&dq=las+5v+del+big+data&ots=_YV7S66e1R&sig=EzoVq0jzhsLIUAcScOUeEVosbbs&redir_esc=y#v=onepage&q=las%205v%20del%20big%20data&f=false

Gómez, Á. Q. (2021). Análisis de los procesos de tratamiento de información en un estudio de análisis de sentimiento utilizando la tecnología de Google. Vivat Academia, 41-

55.<https://www.vivatacademia.net/index.php/vivat/article/view/1336>

Escobar Galaburda, D. A. (2022). Optimización de tiempo de ejecución con PySpark de Hadoop de un análisis de sentimientos de

tweets.<https://repositorio.unican.es/xmlui/handle/10902/26886>

Recabal Ávila, J. A. (2024). Aplicación de técnicas de Big Data para el procesamiento de datos de la operación del sector

eléctrico.<https://repositorio.uchile.cl/handle/2250/199959>

Galindo, A. G., Alfaro, A. J. J., & Organiche, E. C. (2024). El potencial de la optimización de decisiones empresariales con Power BI. RICT Revista de investigación científica, tecnologica e innovación, 2(4), 1-9.

<https://revista.ccaitease.com/index.php/ridt/article/view/86>